


# UNIVERSITI TEKNIKAL MALAYSIA MELAKA

## FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

# W O R K S H O P 1

# R E P O R T

| | |
|---|---|
| **Name** | : WAN NUR INSYIRAH BINTI WAN MOHD ALI |
| **Matric Number** | : B032410553 |
| **Program** | : BITS S1G2 |
| **Project Title** | : HOTEL RESERVATION SYSTEM |
| **Supervisor Name** | : TS. MASLITA BINTI ABD AZIZ |
| **Evaluator Name** | : TS. DR. LIZAWATI BINTI SALAHUDDIN |

## EXECUTIVE SUMMARY

The Hotel Reservation Management System is developed to help hotels manage room bookings, guest information, and staff operations more efficiently. In many hotels, the reservation process is still handled manually or through third-party booking platforms, which often lead to problems such as double bookings, missing guest data, high commission fees, and delays in processing reservations. These issues make hotel management time-consuming, inaccurate, and less organized.

The main objective of this project is to design and develop a computerized system that automates the hotel's reservation and management process. The system aims to reduce human errors, improve data accuracy and provide a more convenient experience for both guests and staff. It also supports hotel staff in monitoring operations and generating detailed reports for better decision-making.

The system consists of five key modules which is Login and Register, Module Management Module, Reservation Module, Report Module and Reward Module. Guests can register, log in and make room reservations online. Staff members can update room details, confirm bookings and manage reports, while the staff oversees all operations to ensure smooth performance and data accuracy.

This project is significant because it enhances hotel efficiency, minimizes manual tasks, and ensures secure handling of hotel data. It also improves customer satisfaction through a faster, more reliable booking process and supports the hotel's digital transformation. By implementing this system, hotels can operate more effectively, save time, and provide better services to their customers

# TABLE OF CONTENTS

**PAGE**

**EXECUTIVE SUMMARY ..... i**

**TABLE OF CONTENTS ..... 1**

**CHAPTER 1: INTRODUCTION ..... 4**

1.1 Introduction ..... 4

1.2 Problem Statement ..... 4

1.3 Objective (s) of the project ..... 4

1.4 Scope ..... 5

1.4.1 Modules to be developed ..... 5

1.4.2 Target User ..... 6

1.5 Project Significance ..... 6

1.6 Gantt Chart of Project Activities ..... 6

**CHAPTER 2: ANALYSIS OF PROBLEM ..... 7**

2.1 Problem Decomposition Description ..... 7

2.2 Structured Chart ..... 8

**CHAPTER 3: DESIGN ..... 9**

3.1 Flowchart ..... 9

3.2 ERD....23

3.3 Data Dictionary....24

USER....24

STAFF....24

CUSTOMER....24

ROLE....25

REWARD....25

ROOM....26

REPORT....26

3.4 Interface Design....27

**CHAPTER 4: IMPLEMENTATION....40**

4.1 Naming Convention....40

4.2 Function....41

4.3 Array....42

4.4 Selection....42

4.5 Control ..... 43

4.6 Pointer ..... 44

4.7 Error Handling ..... 45

4.8 Business Rule Implementation ..... 47

4.9 Complex Calculation Implementation ..... 48

4.10 Report Generation for Analysis Implementation ..... 50

**CHAPTER 5: CONCLUSION ..... 51**

5.1 Constraints ..... 51

5.2 Future Improvements ..... 51

**REFERENCES ..... 52**

# CHAPTER 1: INTRODUCTION

## 1.1 Introduction

The Hotel Reservation Management System is developed to help hotels manage their operations more efficiently. Many small and medium-sized hotels still use manual or third-party booking methods, which often cause problems like double bookings, missing guest information, and slow updates. This system aims to solve those issues by providing a digital platform for room bookings, guest records, and staff management. Guests can register, log in, and make reservations easily, while staff can manage rooms, update details, and generate reports. The project aims to reduce manual work, improve data accuracy, and enhance customer satisfaction through a more modern and organized hotel management process.

## 1.2 Problem Statement

Many hotels still use manual methods or third-party platforms to manage room bookings and guest information, which often cause issues such as double bookings, missing data, and slow updates. These problems make hotel management inefficient and time-consuming, as staff spend extra time handling records while guests experience inconvenience during booking or check-in. To overcome these challenges, a computerized hotel reservation system is needed to make the process faster, more accurate, and well-organized. This system helps reduce human errors, improve data management, and enhance customer satisfaction by providing a smoother and more reliable booking experience.

## 1.3 Objective (s) of the project

This project embarks on the following objectives:

1. To implement five modular functionalities with CRUD database management for the Hotel Reservation System:
    - Login and Register Module
    - Management Module
    - Reservation Module
    - Report Module

- Reward Module

2. To perform data validation and checking to prevent double bookings, missing records, and incorrect room availability.

3. To develop a report generator for data analysis purposes, allowing staff to view booking statistics, revenue summaries, and hotel room performance reports for The Hotel Reservation System.

## 1.4 Scope

### 1.4.1 Modules to be developed

The system will consist of the following modules:

- Login and Register Module
  – allows guests, staff, and manager to log in and register to the system.

- Reservation Module
  – enables guests to make, update, or cancel room bookings.

- Management Module
  – allows staff to manage room, user and booking.

- Report Module
  – generates reports on bookings, guest data and hotel room performance.

- Reward Module
  – provides a reward system for guests based on their booking activities.

### 1.4.2 Target User

- Guests who want to book rooms online easily.
- Hotel Staff who manage room details, guest information, and daily operations.

## 1.5 Project Significance

The Hotel Reservation Management System serves as a centralized and user-friendly platform designed to streamline the management of reservations, guest information and staff operations. By digitalizing hotel operations, the system supports significantly better efficiency and higher customer satisfaction. It improves the accuracy, speed and organization of daily management tasks while reducing human errors that are typically associated with manual or semi-digital booking methods. Conclusively, the system empowers staff to save considerable time and effort through the implementation of automated processes.

## 1.6 Gantt Chart of Project Activities

| Activities | WEEK | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
| Phase 1: Requirement Gathering | | | | | | | | | | | | | | |
| Confirm Project Title | | | | | | | | | | | | | | |
| Defined Project | | | | | | | | | | | | | | |
| Phase 2: Design | | | | | | | | | | | | | | |
| Create Preliminary Design | | | | | | | | | | | | | | |
| Phase 3: Implement | | | | | | | | | | | | | | |
| Create function login | | | | | | | | | | | | | | |
| Create user management function | | | | | | | | | | | | | | |
| Create reservation function | | | | | | | | | | | | | | |
| Create reward function | | | | | | | | | | | | | | |
| Create generate reports function | | | | | | | | | | | | | | |
| Build reservation page (all function) | | | | | | | | | | | | | | |
| Phase 4: Testing | | | | | | | | | | | | | | |
| Check for every bugs and error occurred | | | | | | | | | | | | | | |
| Phase 5: Demonstration | | | | | | | | | | | | | | |
| Final Presentation | | | | | | | | | | | | | | |

Figure 1.6. Gantt Chart of Project

# CHAPTER 2: ANALYSIS OF PROBLEM

## 2.1 Problem Decomposition Description

To address the complex problem of managing hotel operations and reservations effectively, we decompose it into simpler sub-problems, each focused on a specific area of functionality.

First, User Authentication and Access Control involve managing the registration and login processes for guests and staff. This ensures that each user is assigned a specific role (Customer, Staff, or Manager) and can only access the features relevant to their permissions.

Next, Room Inventory Management focuses on maintaining the availability and details of hotel rooms. This sub-problem handles adding new room types, updating pricing, and managing room statuses such as 'Available', 'Full', or 'Maintenance' to ensure the system reflects real-time room conditions.

The Reservation and Booking Management sub-problem addresses the core functionality of the system, allowing guests to view available rooms and make, update, or cancel bookings. It involves calculating stay durations, applying relevant discounts, and ensuring that the room status is synchronized automatically upon booking.

Reward and Loyalty Tracking involves managing a point-based system for frequent guests. This sub-problem tracks customer points and membership tiers (Bronze, Silver, Gold), allowing for automated discount redemptions and enhancing customer retention.

Finally, Report Generation and Data Analysis involves aggregating data from across the system to generate performance summaries. This includes financial reports, room popularity rankings, and monthly revenue trends to assist management in making informed decisions.

By breaking the hotel reservation process into these sub-problems, each component can be developed and tested independently, ensuring a robust and well-organized system that improves overall hotel efficiency

## 2.2 Structured Chart

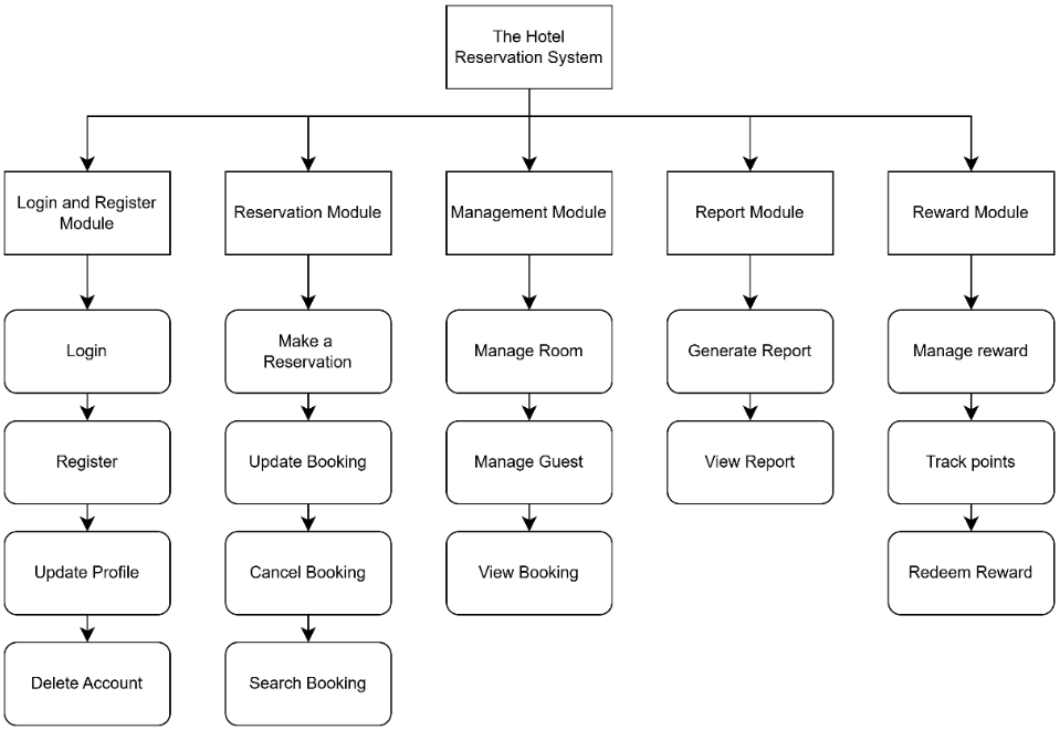



Figure 2.2 Structured Chart

# CHAPTER 3: DESIGN

## 3.1 Flowchart

a. Login

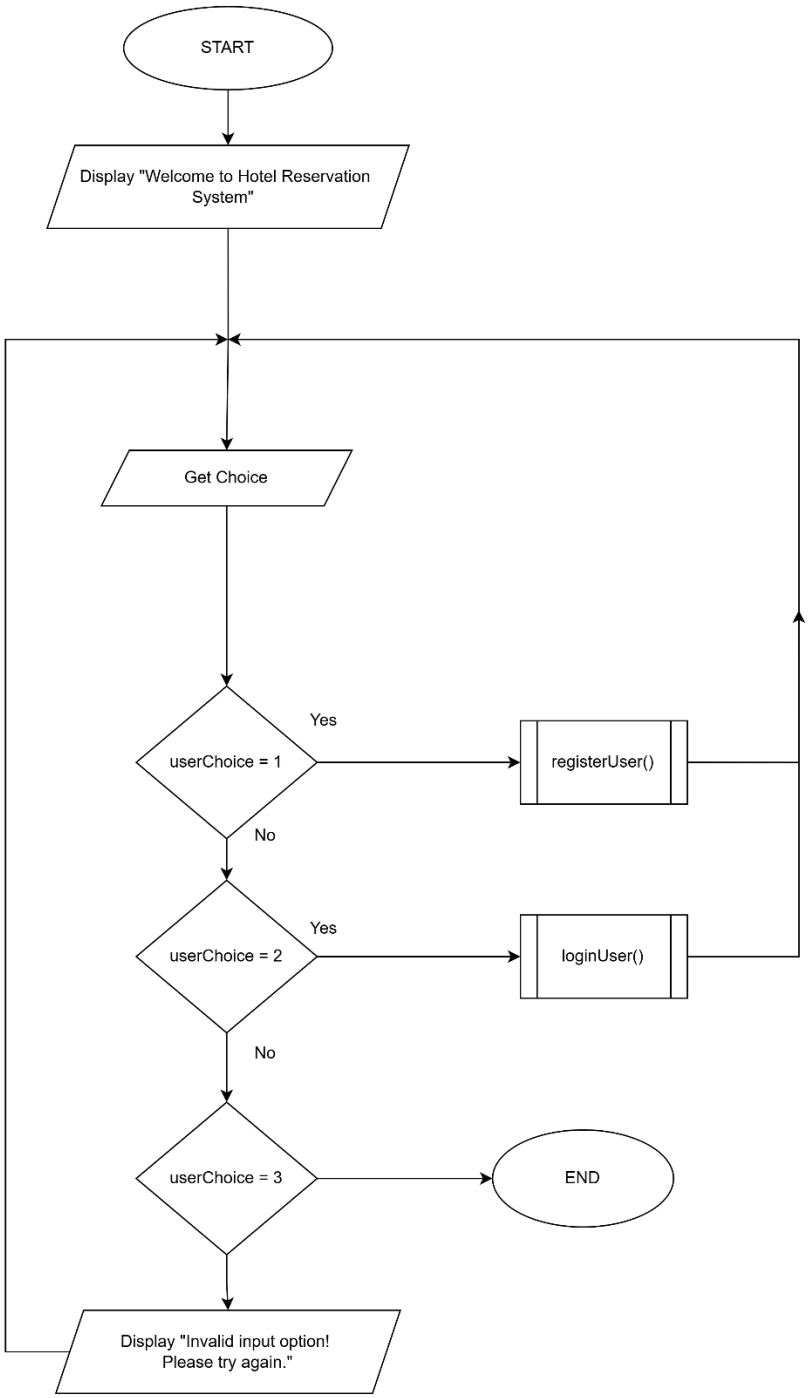



Figure 3.1.1 Login

b. Staff dashboard

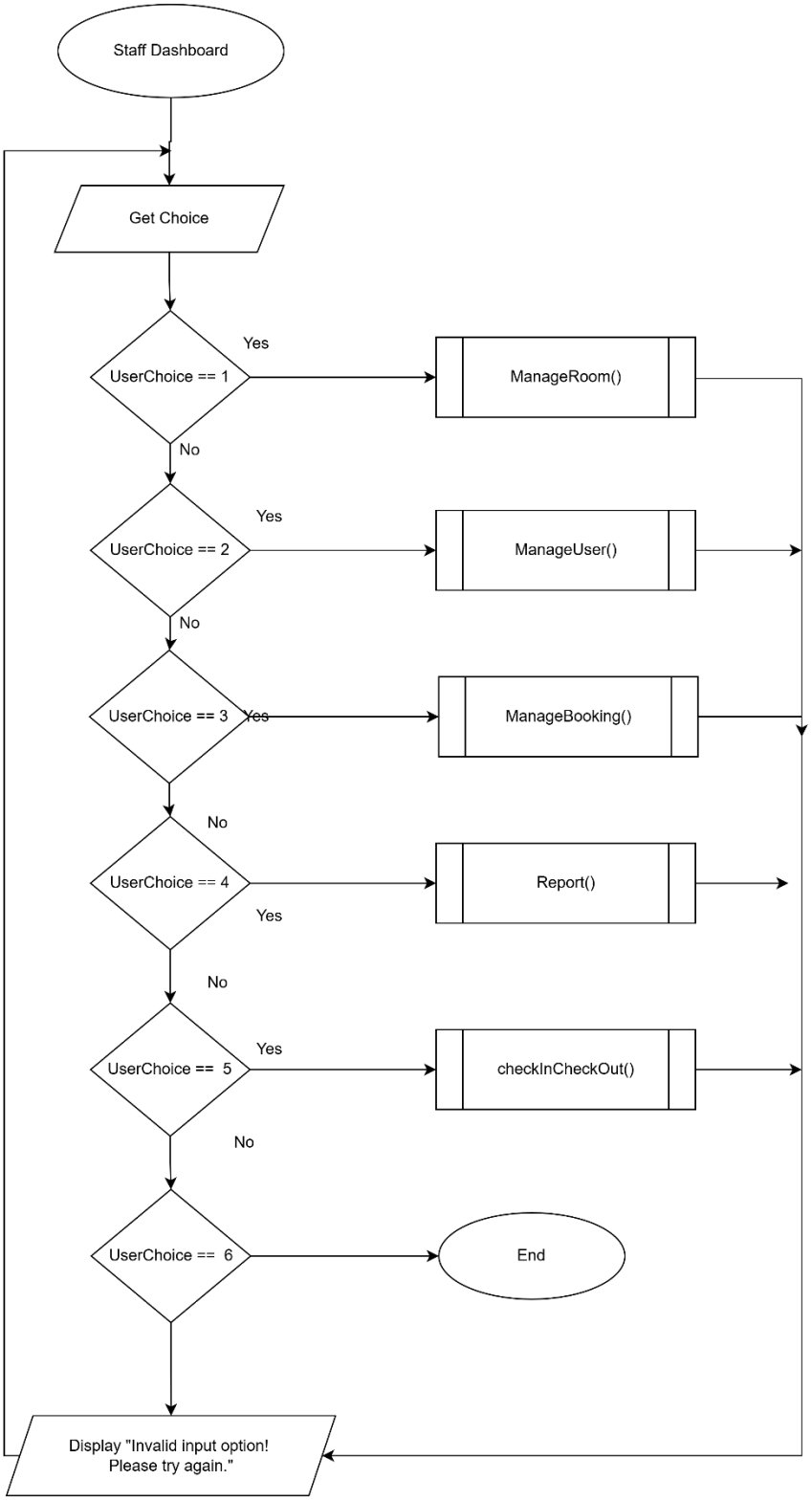



Figure 3.1.2 Staff dashboard

c. Customer dashboard

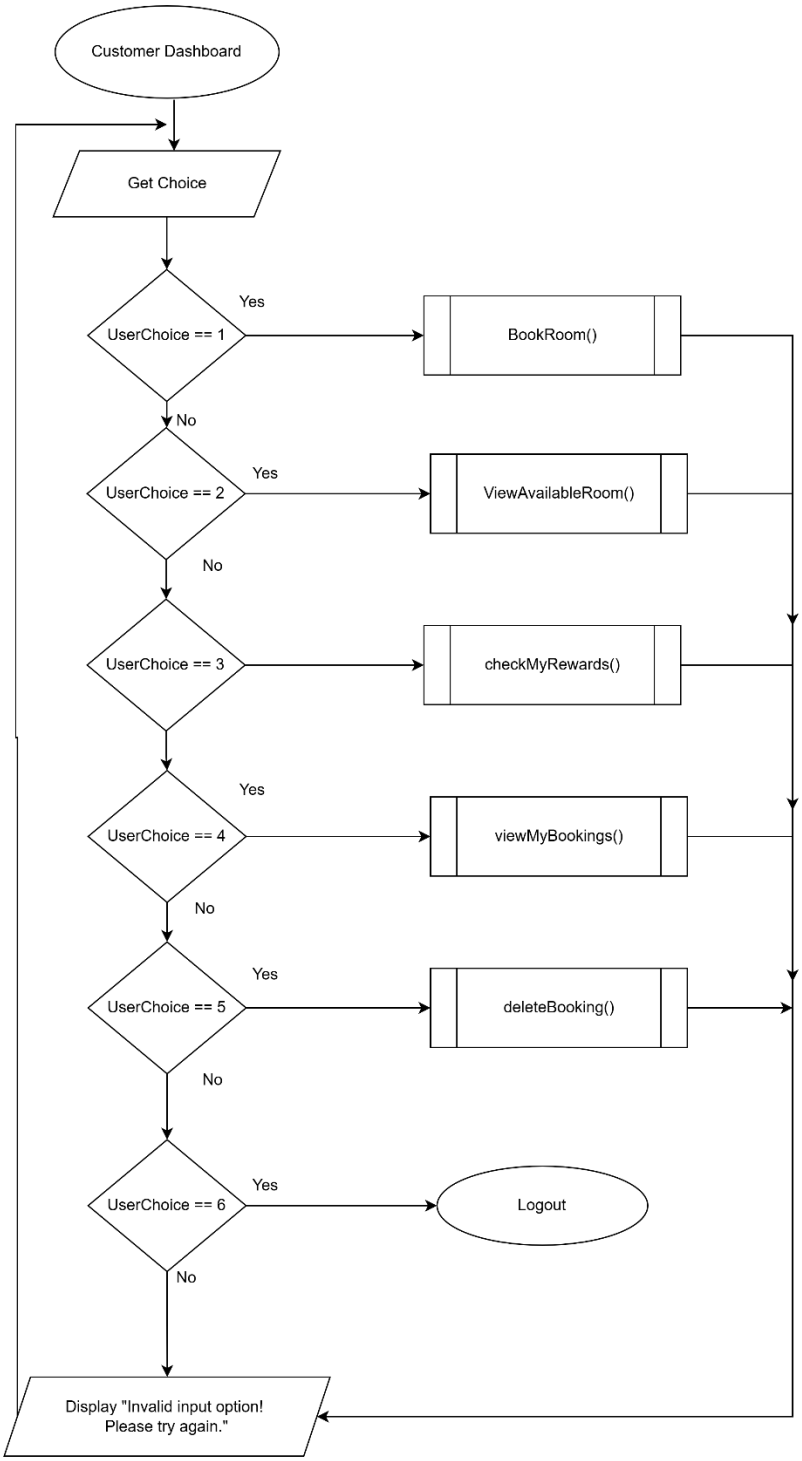



Figure 3.1.3 Customer dashboard

d. Manage Room

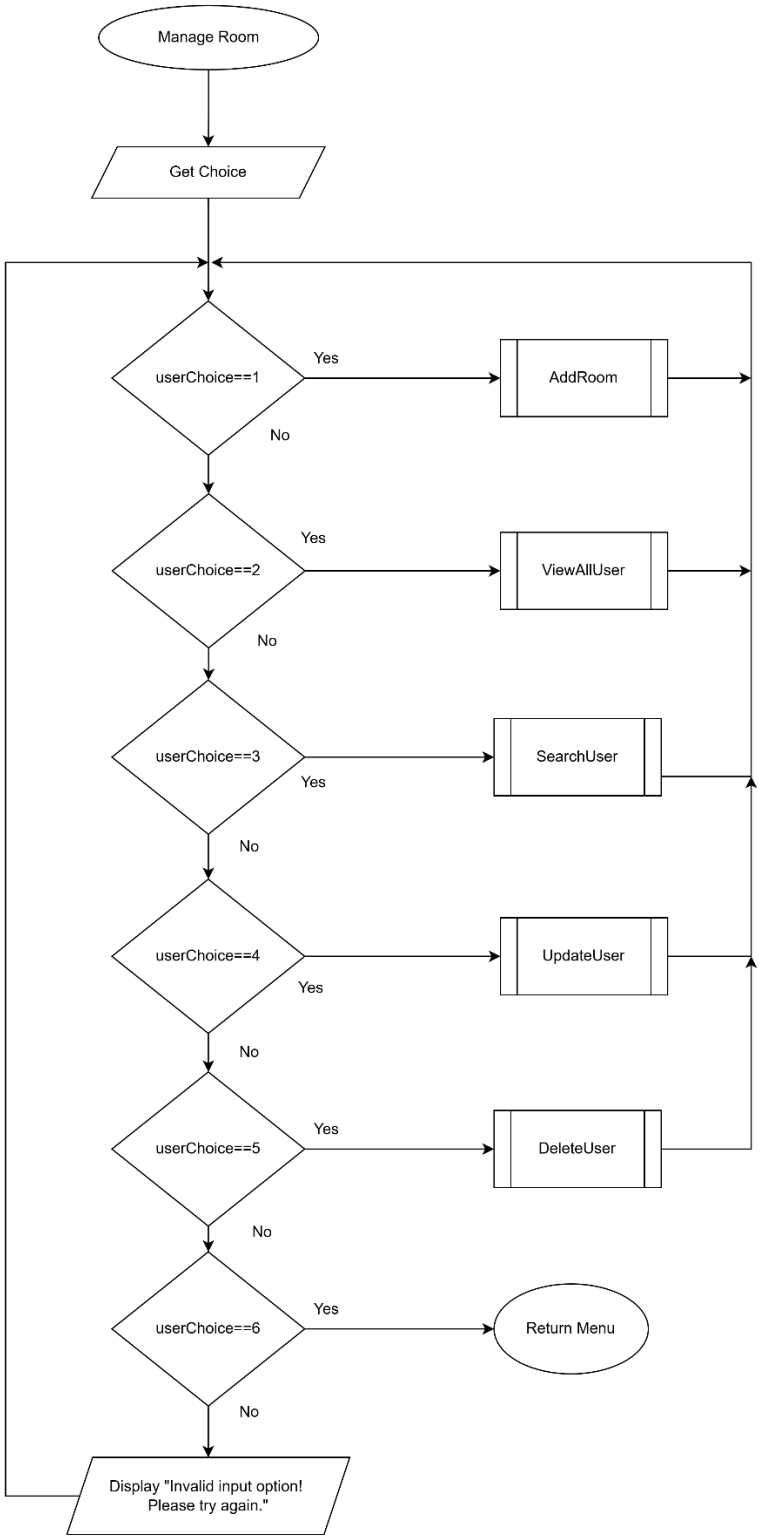



Figure 3.1.4 Manage Room

e. Manage User

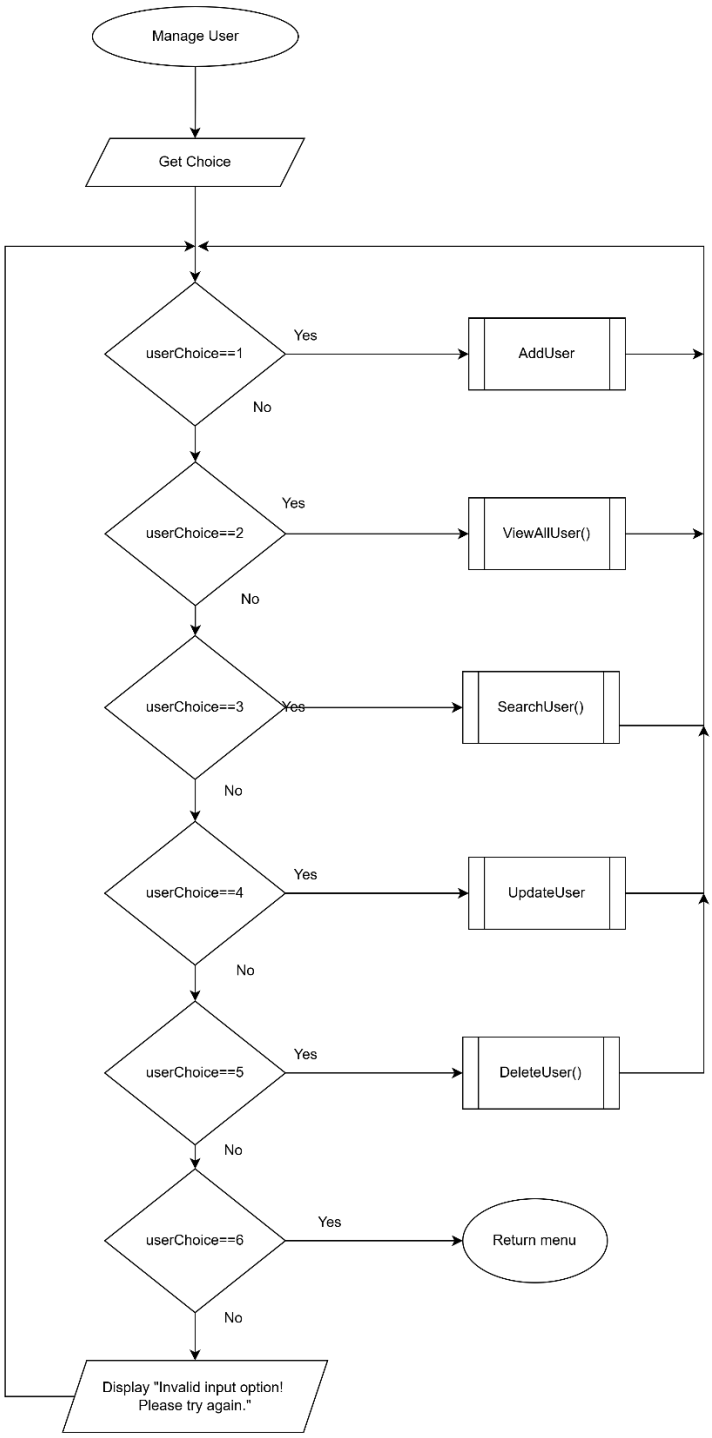



Figure 3.1.5 Manage User

f. Manage Booking

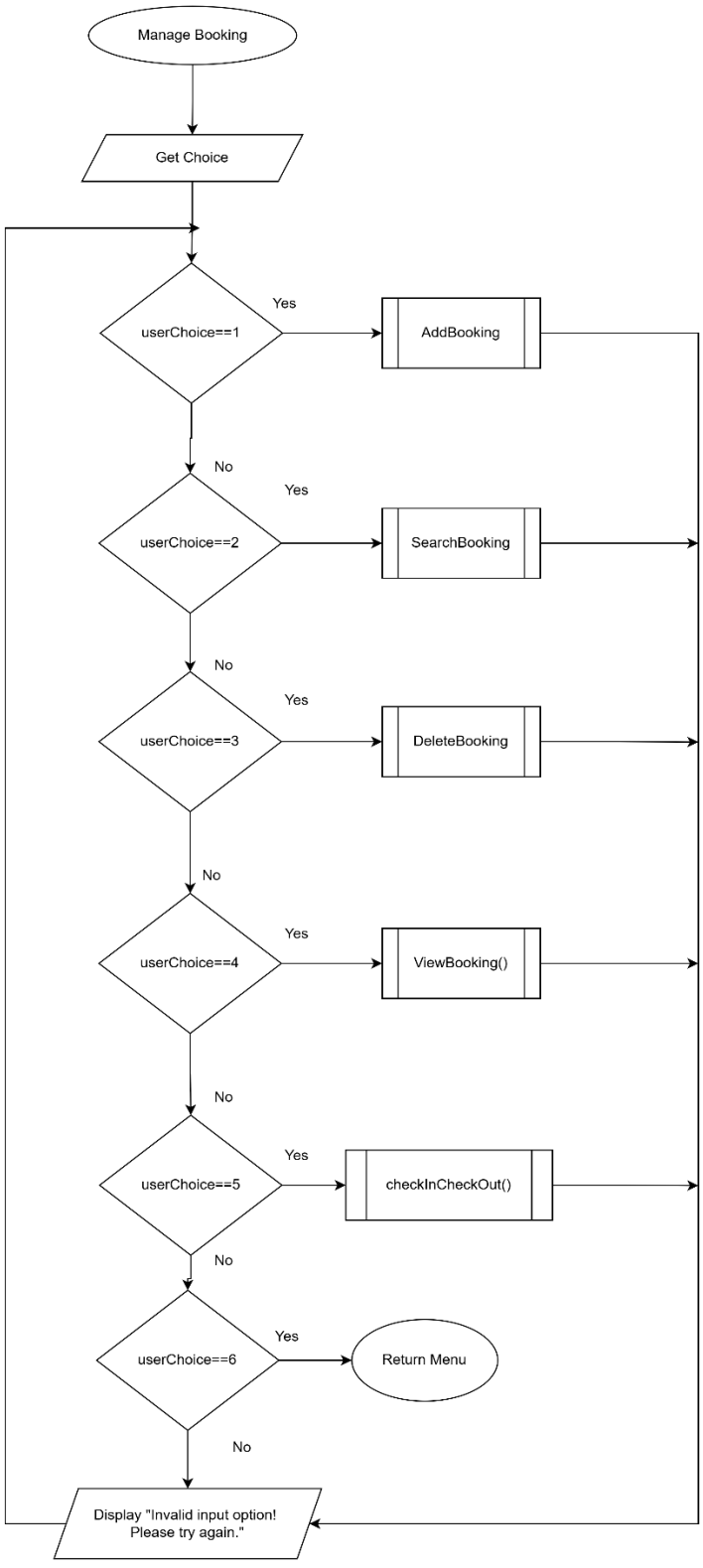



Figure 3.1.6 Manage Booking

g. Book room

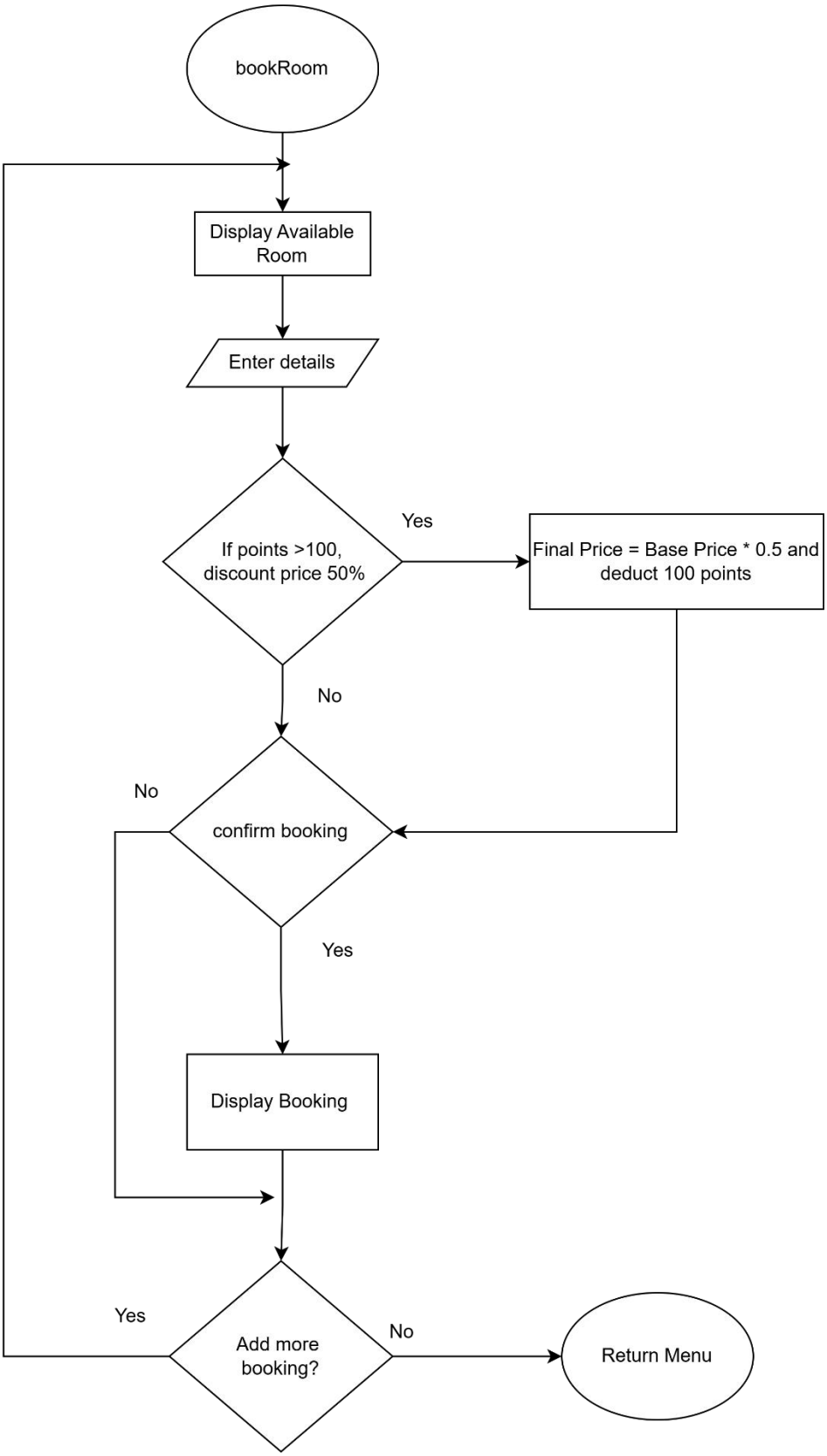



Figure 3.1.7 Book Room and Add Booking

h. Search booking

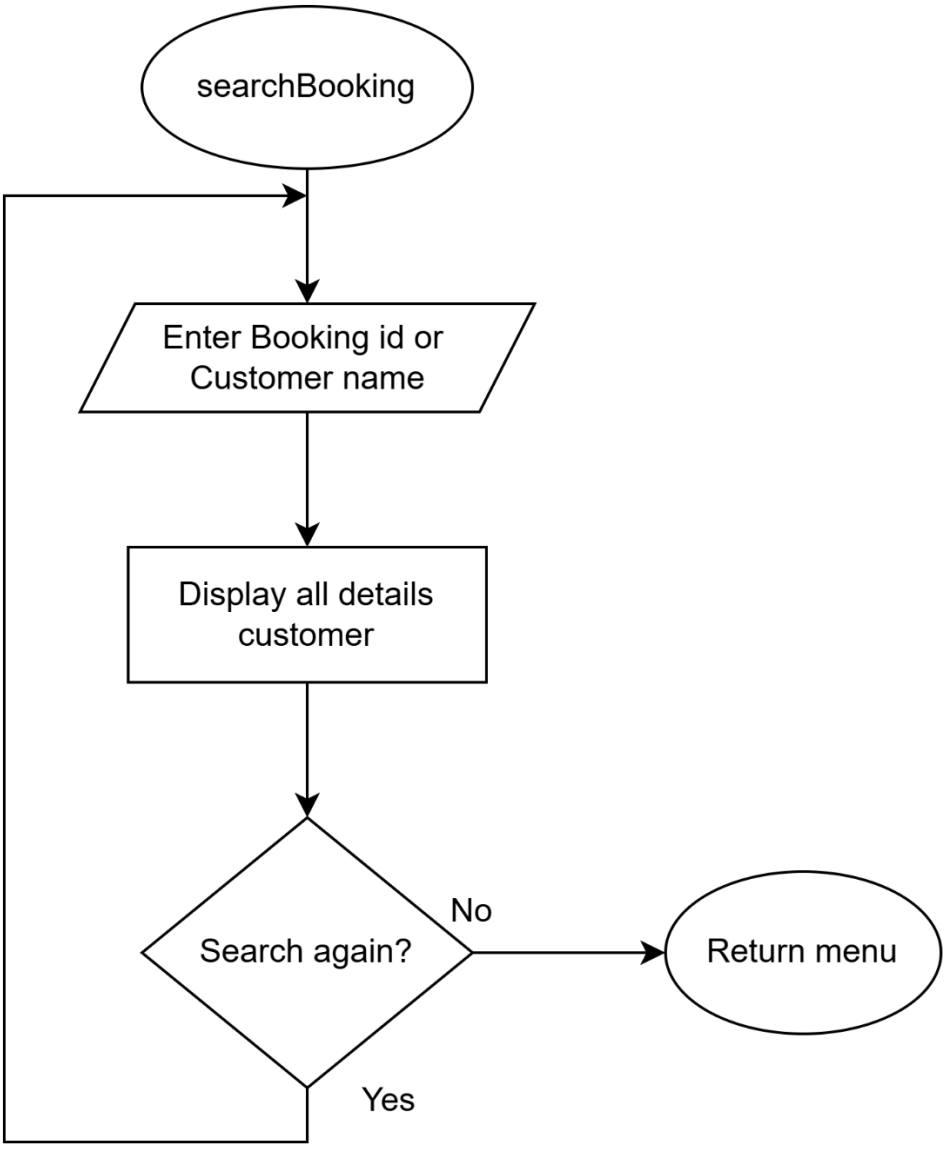



Figure 3.1.8 Search booking

i. Delete Booking

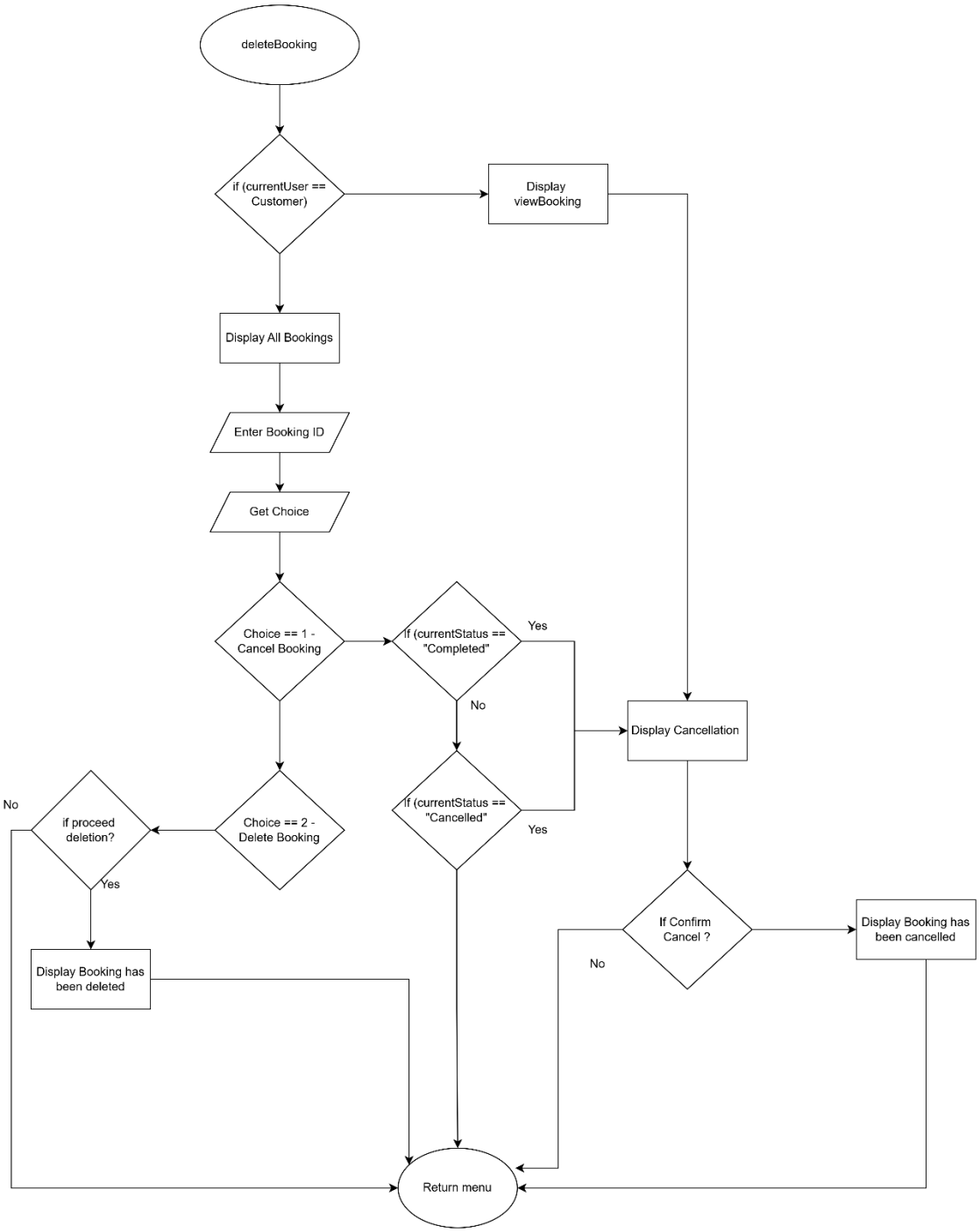



Figure 3.1.9 Delete Booking

j. View Booking

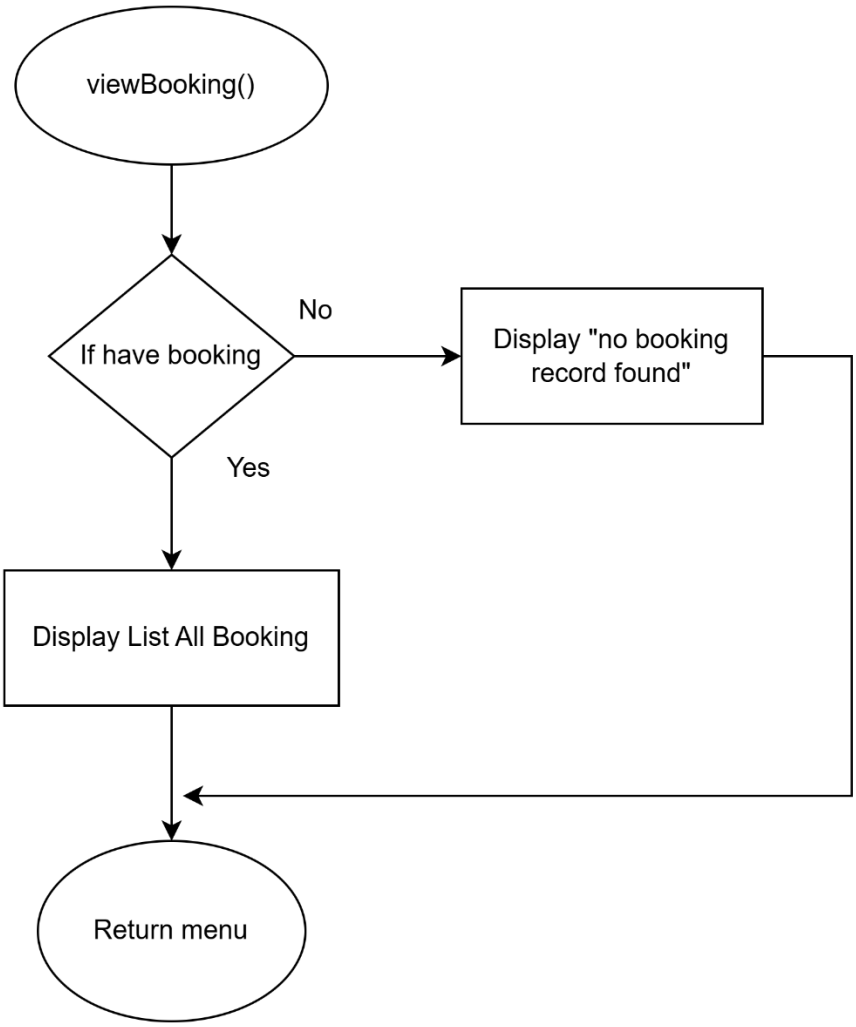



Figure 3.1.10 View Booking

k. Check In Check Out

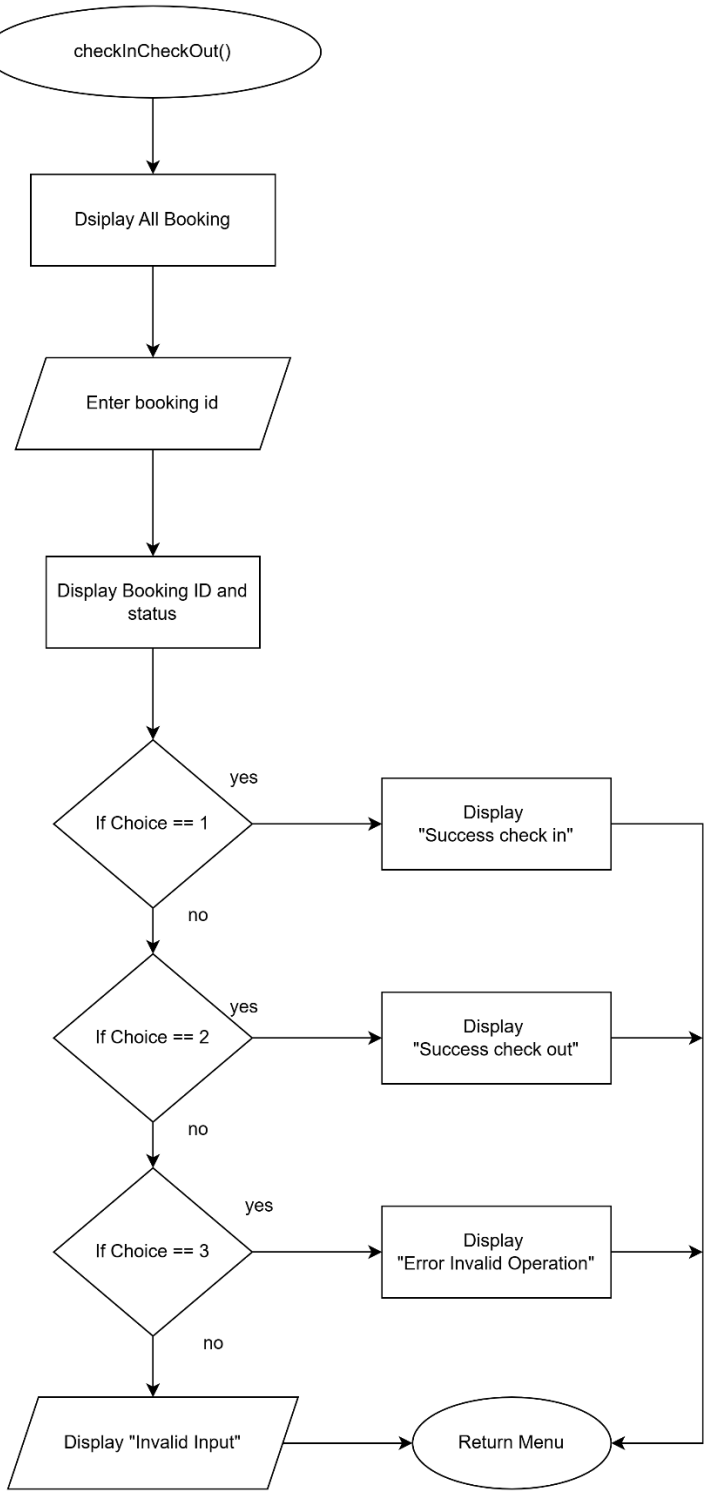



Figure 3.1.11 Check In Check Out

l. View My Booking

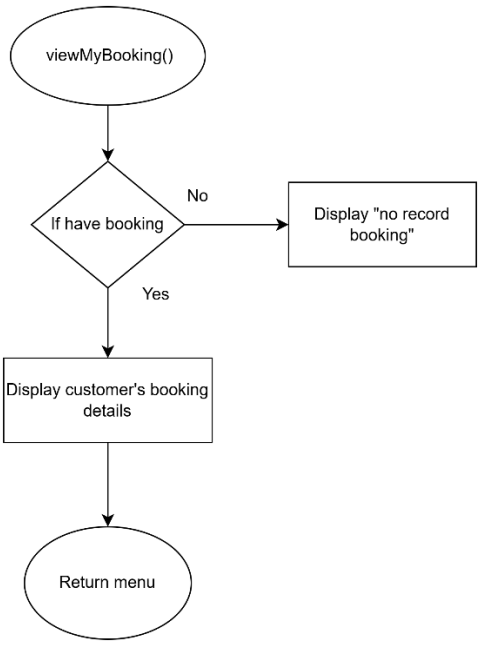



Figure 3.1.12 View My Booking

m. View Available Room

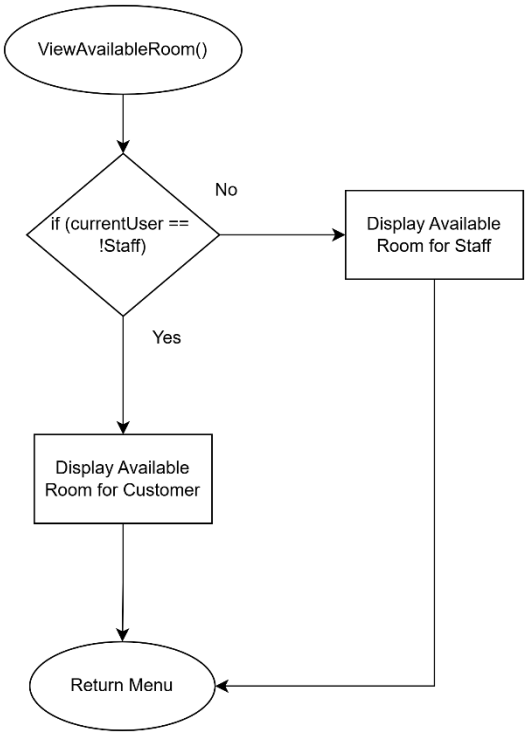



Figure 3.1.13 View Available room

n. Check My Reward

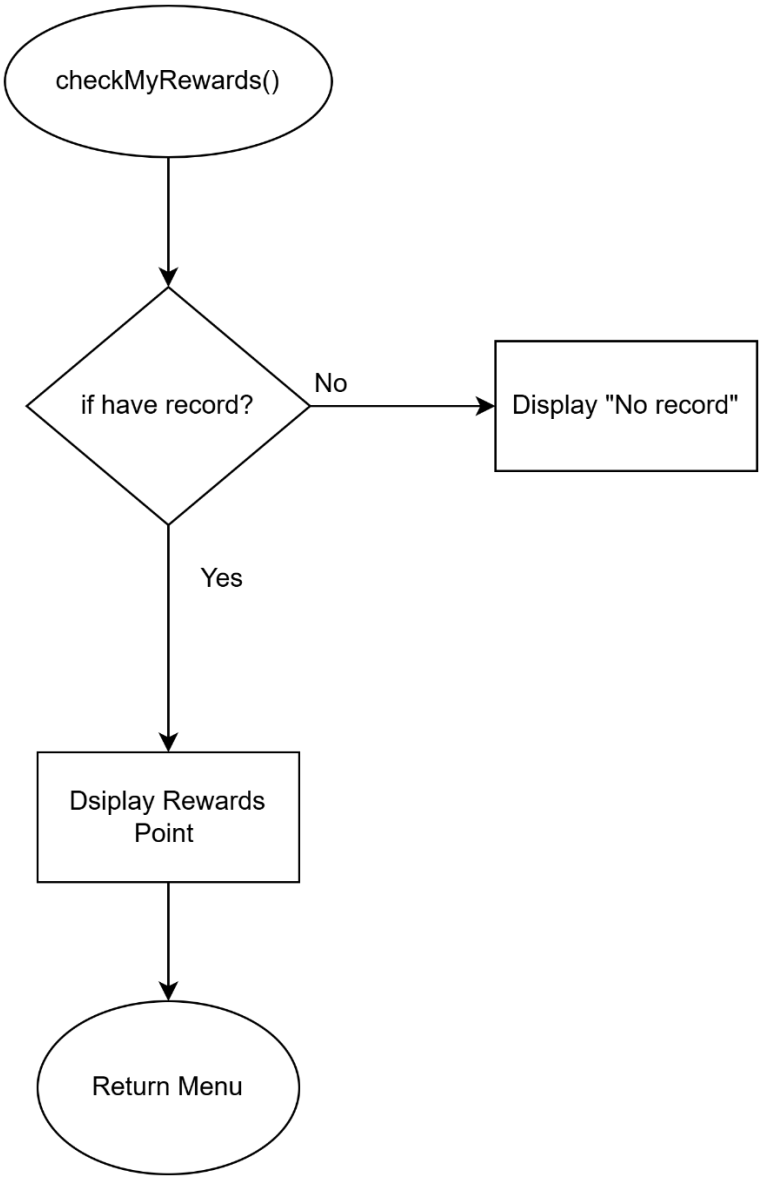



Figure 3.1.14 Check My Reward

o. Generate Report

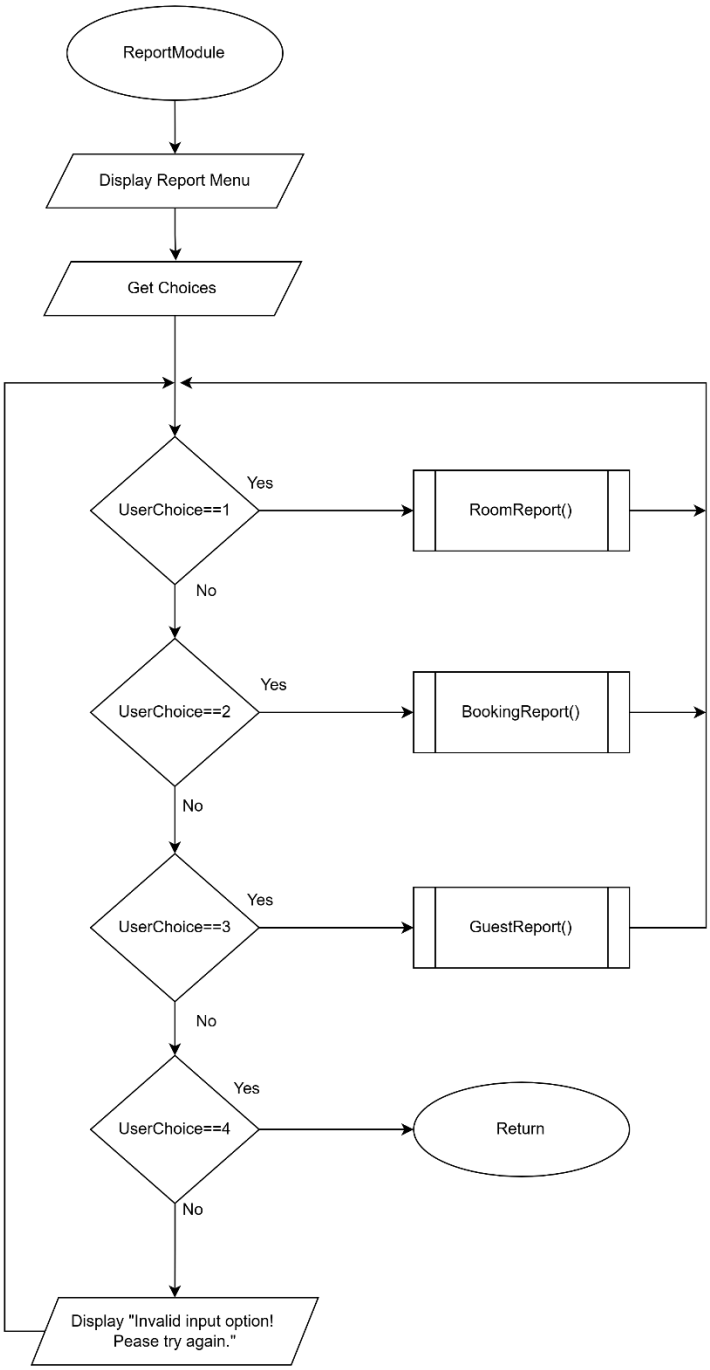



Figure 3.1.15 Generate Report

## 3.2 ERD

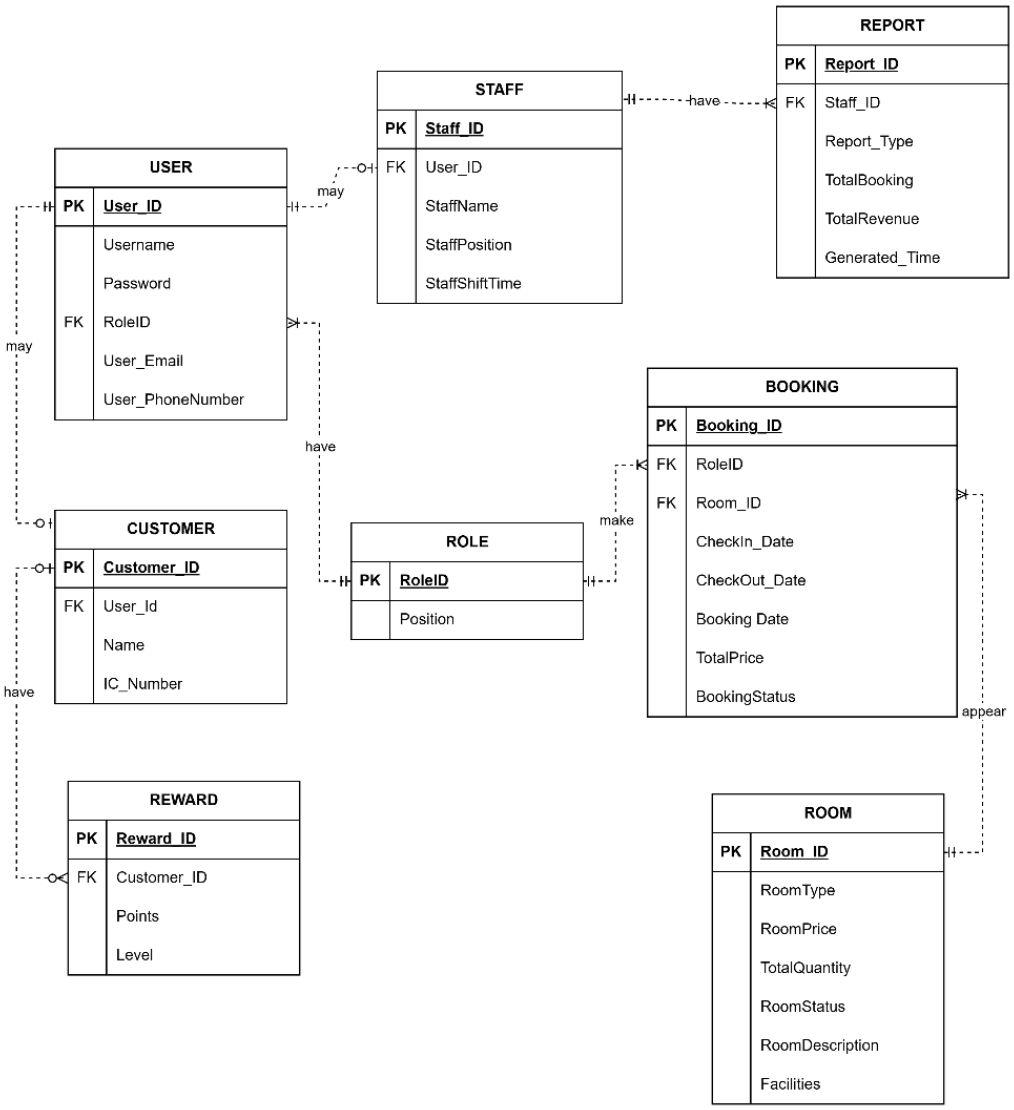



Figure 3.2 ERD

## 3.3 Data Dictionary

### USER

| ATTRIBUTE NAME | DATA TYPE AND SIZE | RANGE | NOT NULL | DEFAULT VALUE | UNIQUE | PK OR FK | FK REFERENCE TABLE |
|---|---|---|---|---|---|---|---|
| **User_ID** | VARCHAR (100) | | YES | | YES | PK | |
| **Username** | VARCHAR(100) | | YES | | YES | | |
| **Password** | VARCHAR(100) | | YES | | YES | | |
| **RoleID** | VARCHAR(100) | | YES | | | FK | ROLE |
| **User_Email** | VARCHAR(100) | | YES | | YES | | |
| **User_PhoneNumber** | VARCHAR(20) | | YES | | | | |

Table 3.3.1 Table User

### STAFF

| ATTRIBUTE NAME | DATA TYPE AND SIZE | RANGE | NOT NULL | DEFAULT VALUE | UNIQUE | PK OR FK | FK REFERENCE TABLE |
|---|---|---|---|---|---|---|---|
| **Staff_ID** | VARCHAR (100) | | YES | | YES | FK | |
| **User_ID** | VARCHAR(100) | | YES | | YES | FK | |
| **StaffName** | VARCHAR(100) | | YES | | YES | | |
| **StaffPosiotion** | VARCHAR(100) | | YES | | | F | ROLE |
| **StaffPosition** | VARCHAR(50) | | YES | | YES | | |
| **StaffSHiftName** | VARCHAR(50) | | YES | | | | |

Table 3.3.2 Table Staff

### CUSTOMER

| ATTRIBUTE NAME | DATA TYPE AND SIZE | RANGE | NOT NULL | DEFAULT VALUE | UNIQUE | PK OR FK | FK REFERENCE TABLE |
|---|---|---|---|---|---|---|---|
| **Customer_ID** | VARCHAR (100) | | YES | | YES | FK | |
| **User_ID** | VARCHAR(100) | | YES | | YES | FK | UPSR |
| **Name** | VARCHAR(100) | | YES | | YES | | |
| **Ic_Number** | VARCHAR(100) | | YES | | | F | |

Table 3.3.3 Table Customer

## ROLE

| ATTRIBUTE NAME | DATA TYPE AND SIZE | RANGE | NOT NULL | DEFAULT VALUE | UNIQUE | PK OR FK | FK REFERENCE TABLE |
|---|---|---|---|---|---|---|---|
| **RoleID** | VARCHAR (100) | | YES | | YES | PK | |
| **Position_ID** | VARCHAR(100) | | YES | | YES | FK | |

Table 3.3.4 Table Role

## REWARD

| ATTRIBUTE NAME | DATA TYPE AND SIZE | RANGE | NOT NULL | DEFAULT VALUE | UNIQUE | PK OR FK | FK REFERENCE TABLE |
|---|---|---|---|---|---|---|---|
| **Reward_ID** | VARCHAR (100) | | YES | | | FK | |
| **Customer_ID** | VARCHAR (100) | | YES | | | FK | |
| **StaffNamE** | VARCHAR (100) | | YES | | YES | | |
| **Points** | VARCHAR (100) | | YES | | | | ROLE |
| **Level** | VARCHAR(20) | | YES | | | | |

Table 3.3.5 Table Reward

## BOOKING

| ATTRIBUTE NAME | DATA TYPE AND SIZE | RANGE | NOT NULL | DEFAULT VALUE | UNIQUE | PK OR FK | FK REFERENCE TABLE |
|---|---|---|---|---|---|---|---|
| **Booking_ID** | VARCHAR (100) | | YES | | YES | PK | |
| **RoleID** | VARCHAR (100) | | YES | | | FK | ROLE |
| **Room_ID** | VARCHAR (100) | | | | | FK | ROOM |
| **checkIn_Date** | VARCHAR (100) | | YES | | | | |
| **CheckOut_Date** | VARCHAR (100) | | YES | | | | |
| **Booking_Date** | VARCHAR (50) | | YES | | | | |
| **TotalPrice** | VARCHAR (10) | | YES | | | | |
| **BookingStatus** | VARCHAR (20) | | YES | | | | |

Table 3.3.6 Table Booking

## ROOM

| ATTRIBUTE NAME | DATA TYPE AND SIZE | RANGE | NOT NULL | DEFAULT VALUE | UNIQUE | PK OR FK | FK REFERENCE TABLE |
|---|---|---|---|---|---|---|---|
| **Room_ID** | VARCHAR (100) | | YES | | YES | PK | |
| **RoomTye** | VARCHAR (100) | | YES | | | | ROLE |
| **RoomPrice** | VARCHAR (100) | | | | | | ROOM |
| **checkIn_Date** | VARCHAR (100) | | YES | | | | |
| **TotalQuantity** | VARCHAR (100) | | YES | | | | |
| **Roomdecription** | VARCHAR (50) | | YES | | | | |
| **Room description** | VARCHAR (10) | | YES | | | | |
| **BookingStatus** | VARCHAR (20) | | YES | | | | |

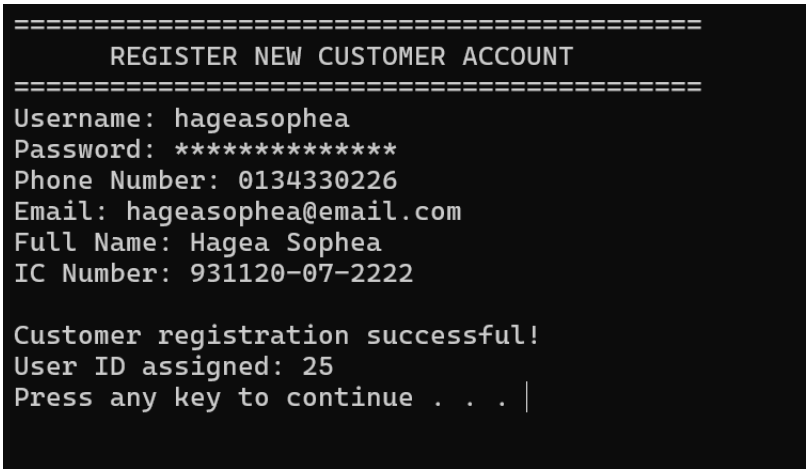
Table 3.3.7 Table Room

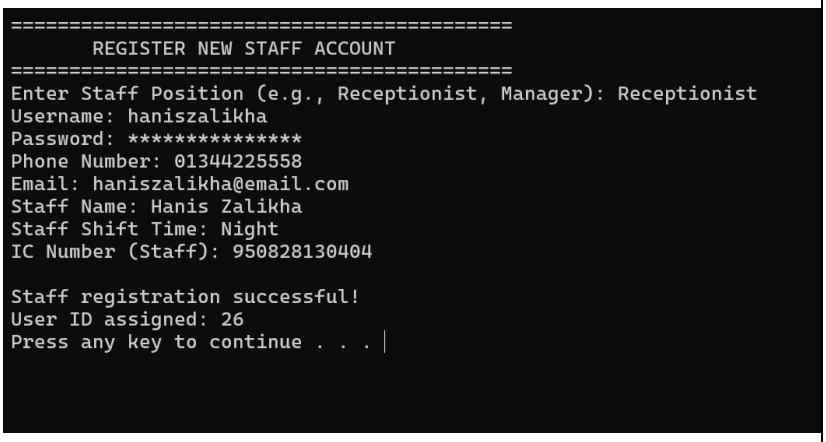
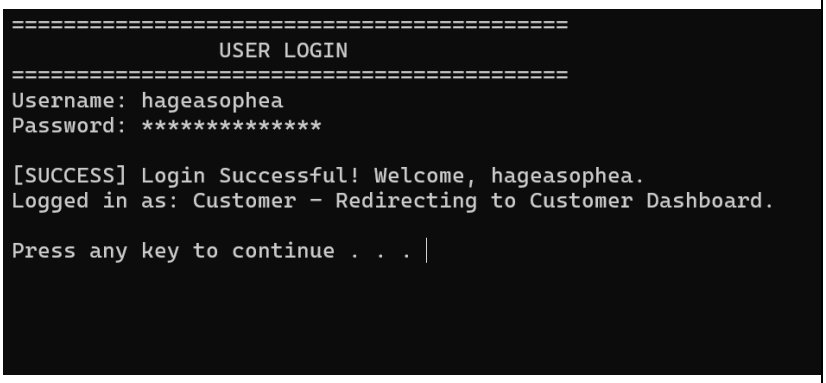
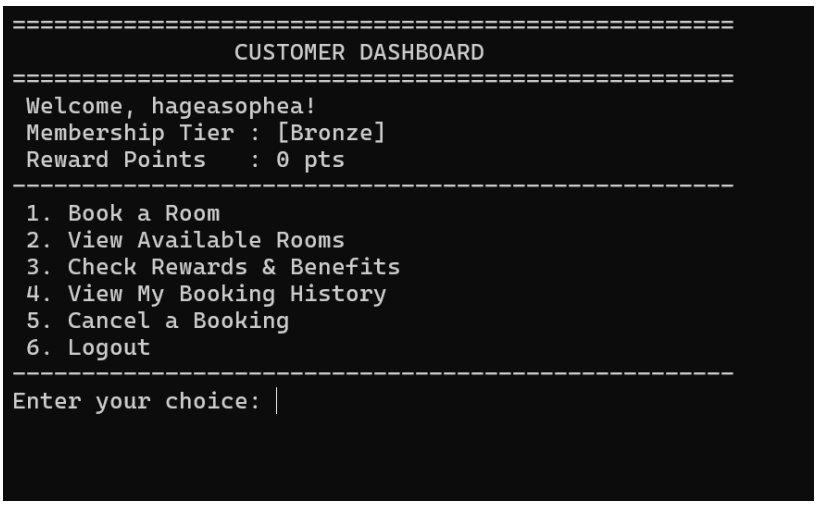
## REPORT

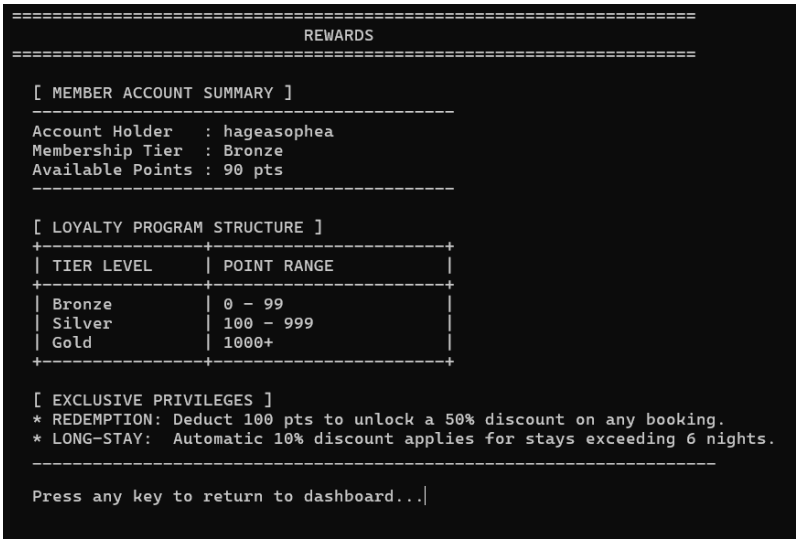
| ATTRIBUTE NAME | DATA TYPE AND SIZE | RANGE | NOT NULL | DEFAULT VALUE | UNIQUE | PK OR FK | FK REFERENCE TABLE |
|---|---|---|---|---|---|---|---|
| **Report ID** | VARCHAR (100) | | YES | | YES | PK | |
| **Staff_ID** | VARCHAR(100) | | YES | | YES | FK | STAFF |
| **Report_Type** | VARCHAR(100) | | YES | | | | |
| **TotalRevenue** | VARCHAR(50) | | YES | | YES | | |
| **StaffSHiftName** | VARCHAR(50) | | YES | | | | |
| **Generated_ilmu** | | | | | | | |

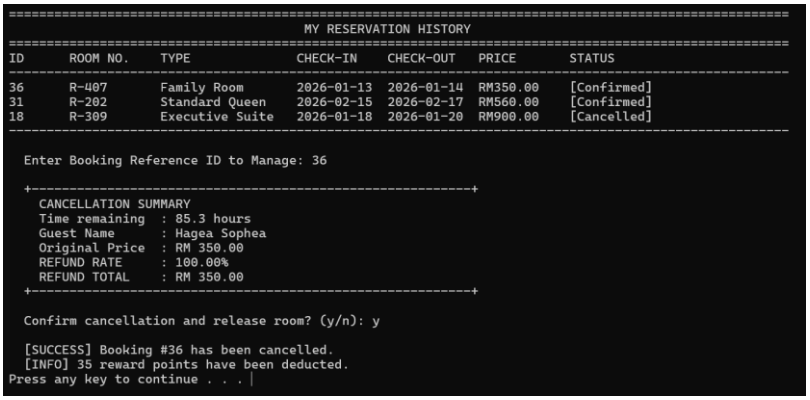
Table 3.3.8 Table Report

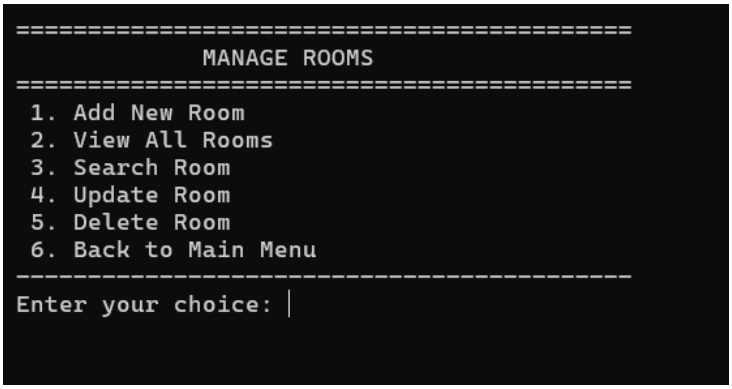
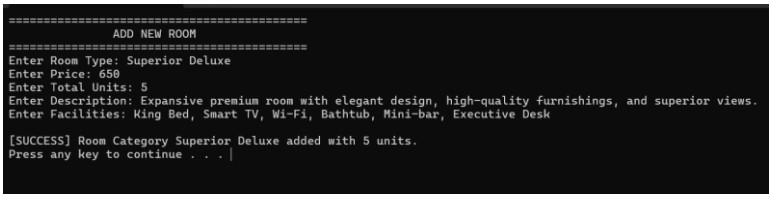
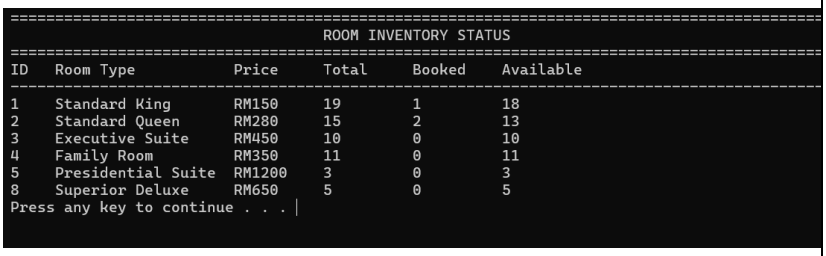
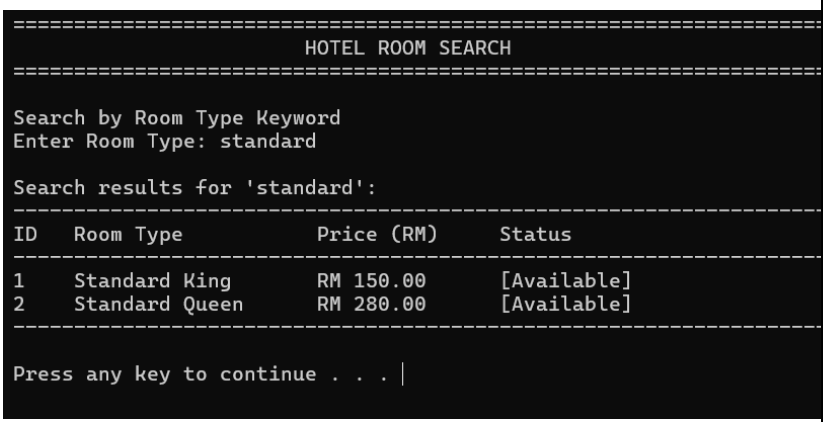
## 3.4 Interface Design

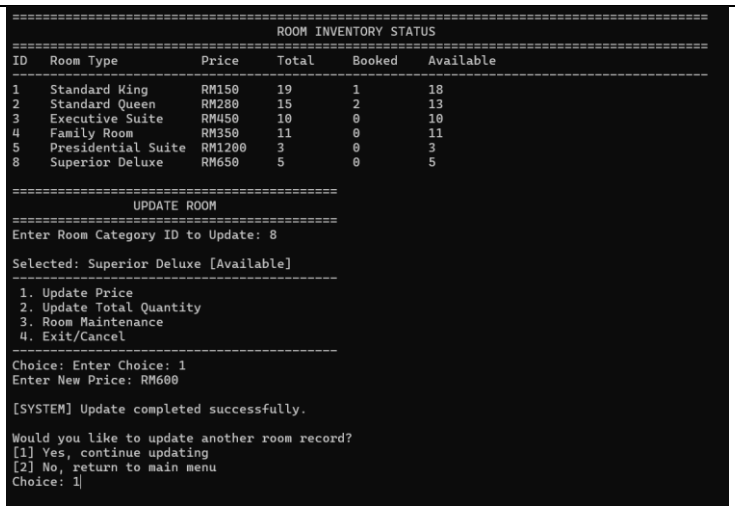
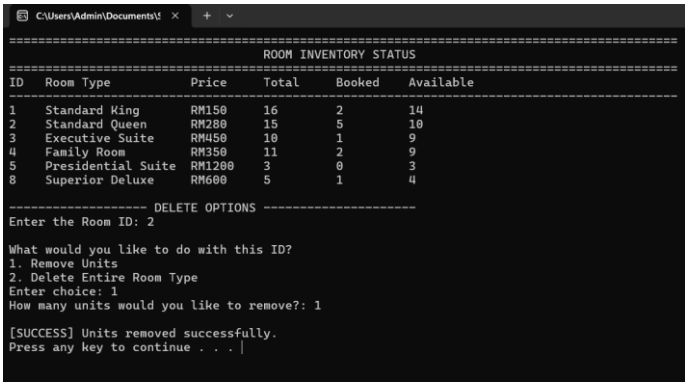
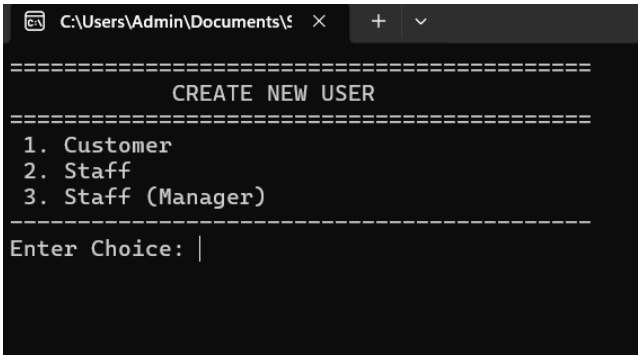
| Figure | | Description |
|---|---|---|
| 3.4.1 |  | Start page when the user opens the system. It shows register and login |
| 3.4.2 |  | Register Menu shows customer and staff registration. |
| 3.4.3 |  | Create new customer at register function which user need to follow to fill in register customer details. |

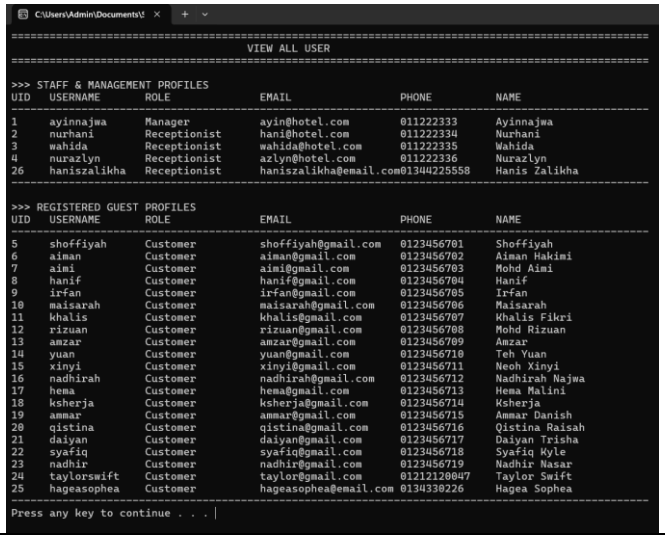
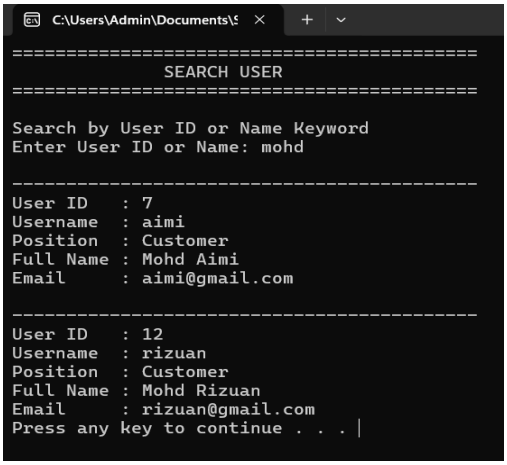
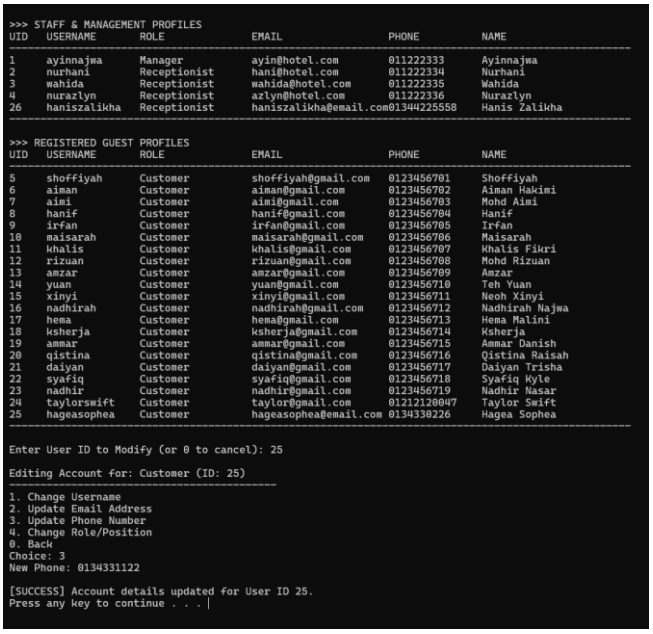
| | | |
|---|---|---|
| 3.4.4 |  | Create new staff at register function which user need to follow to fill in register customer details. |
| 3.4.5 |  | User login shows username and password. User must fill the details, and it will show their specific dashboard based on their role. |
| 3.4.6 |  | Customer dashboard menu. It shows membership and rewards points. Customer can book for their booking. |

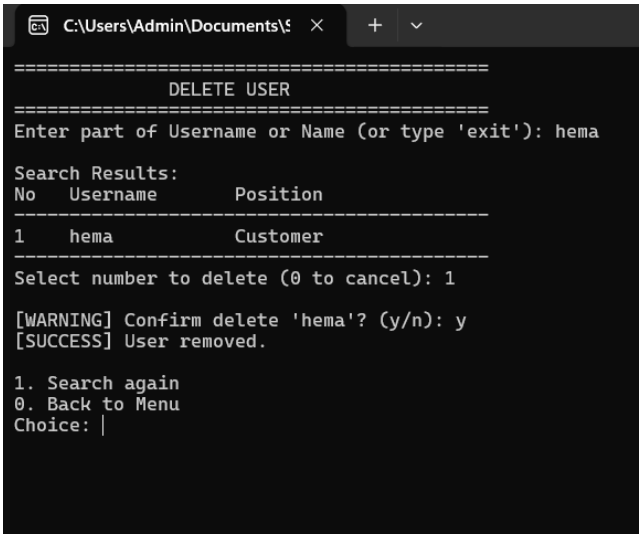
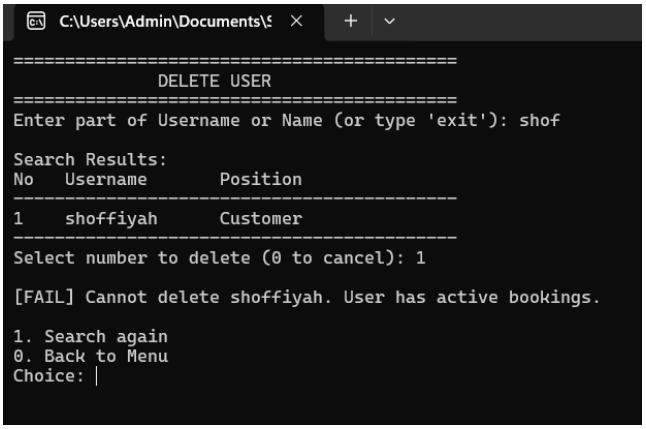
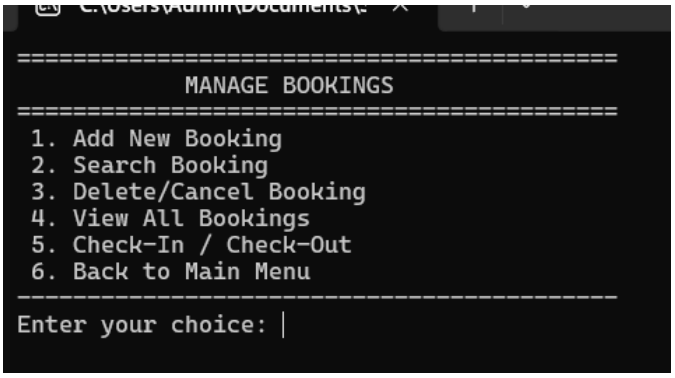
| | | |
|---|---|---|
| 3.4.7 |  | Add book a room function which user need to follow to fill all the details in booking details. |
| 3.4.8 |  | View room function to show all room that available to book. |
| 3.4.9 |  | Check reward function shows reward and membership of the customer. |

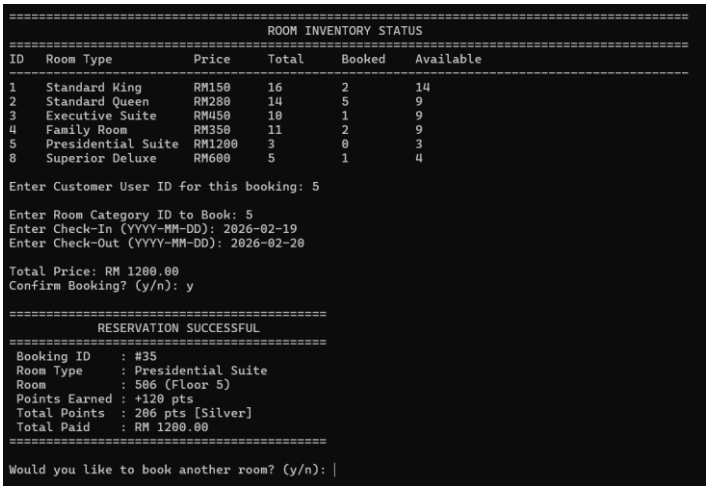
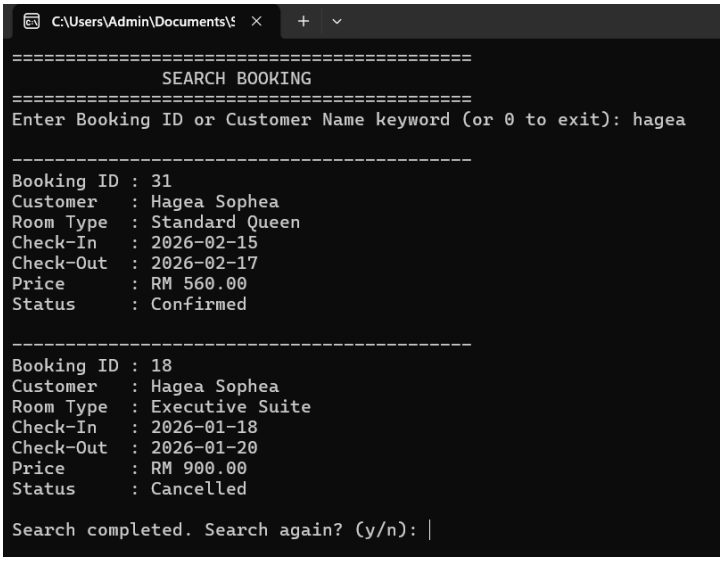
| | | |
|---|---|---|
| 3.4.10 |  | Booking history function for customer to check their booking after book a room. |
| 3.4.11 |  | Cancel booking function for customer who want to cancel booking. Customer will get refund based on time before check in. |
| 3.4.12 |  | Staff dashboard menu shows manage rooms, users, booking and report. |

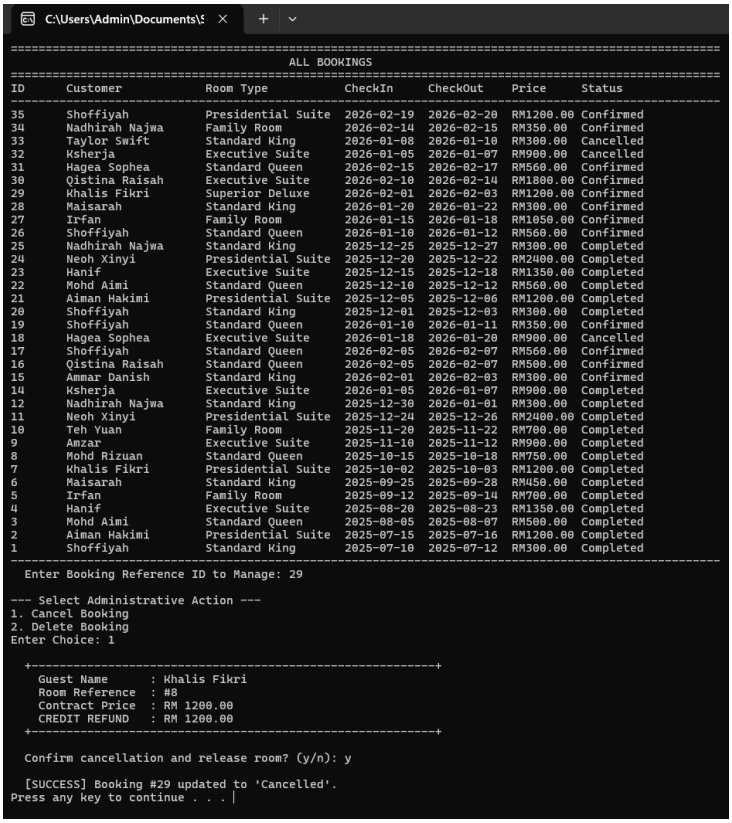
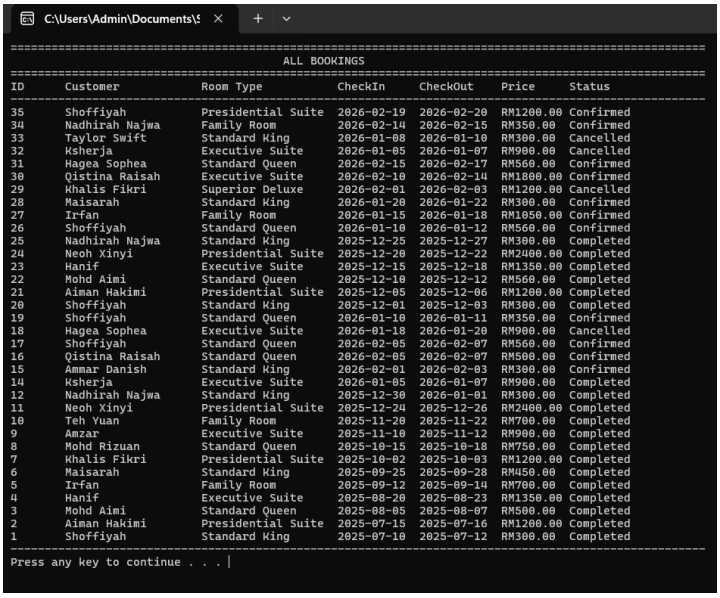
| | | |
|---|---|---|
| 3.4.13 |  | Manage room menu have option for staff to add, view, search, update and delete room. |
| 3.4.14 |  | Add new room which staff need to fill all details about room they want to add. |
| 3.4.15 |  | View room to show all room type, price, total for available and booked. |
| 3.4.16 |  | For search room, staff can verify room availability, current price and status by using keyword-based searching. |

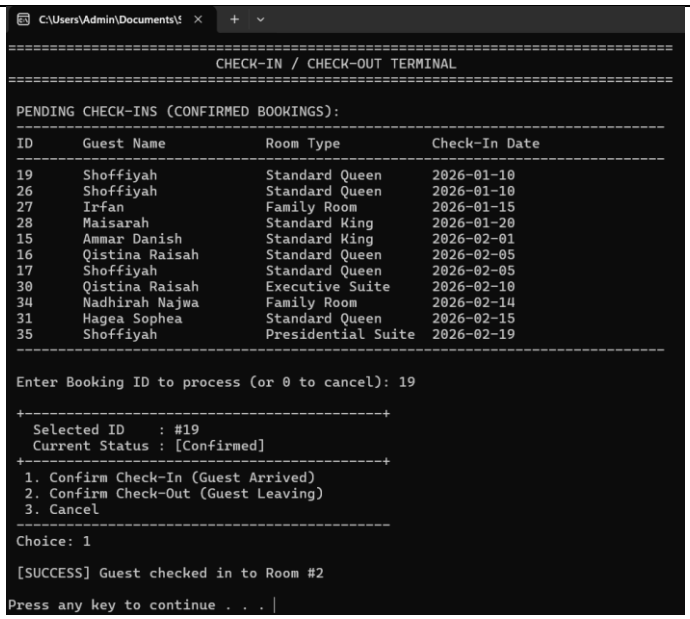
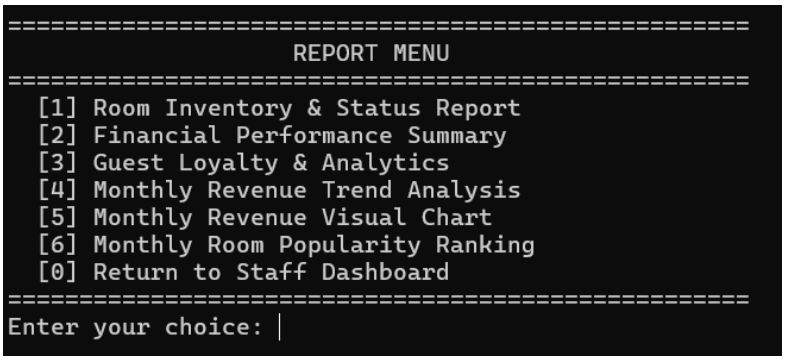
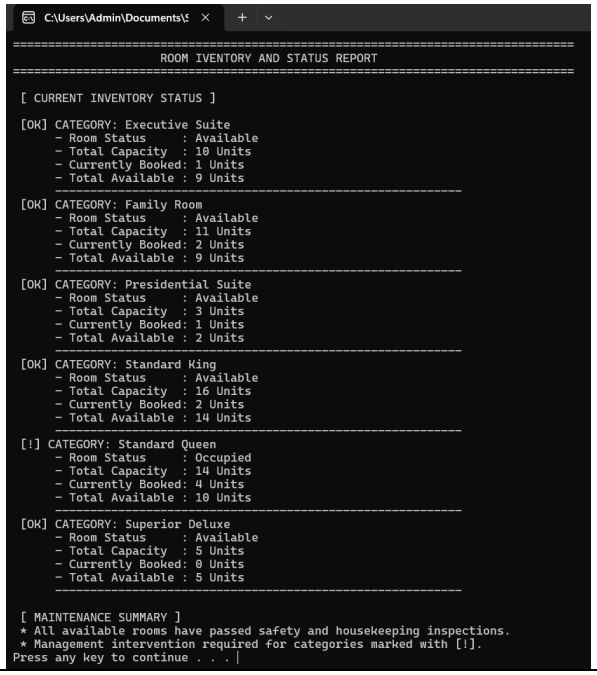
| | | |
|---|---|---|
| 3.4.17 |  | Update room which asks staff to enter room id and show option to update on price, total quantity and room maintenance. |
| 3.4.18 |  | Delete room by entering room id and then it will show option to delete by unit or entire room. |
| 3.4.19 |  | Add user which asks option to staff for create either customer, staff or staff manager. |

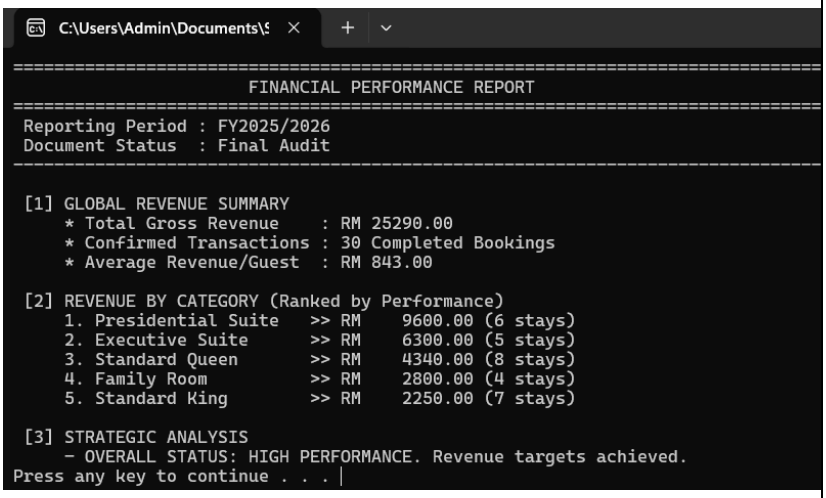
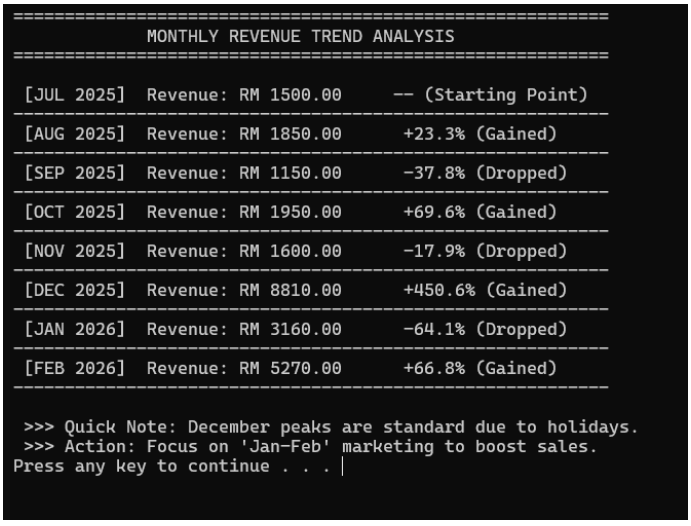
| | | |
|---|---|---|
| 3.4.20 |  | View all user to see all details user which is staff and customer. |
| 3.4.21 |  | For search user, staff can verify user id, username, position, full name and email by adding keyword-based searching. |
| 3.4.22 |  | Update user by entering user id and it will give option to update username, email, phone number and position. |

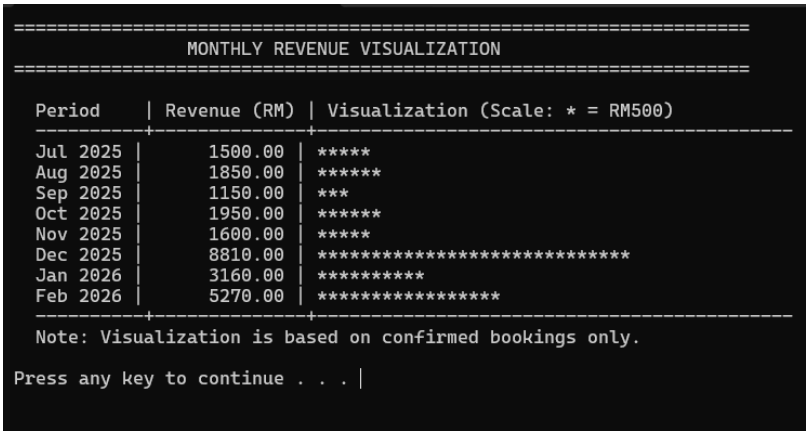
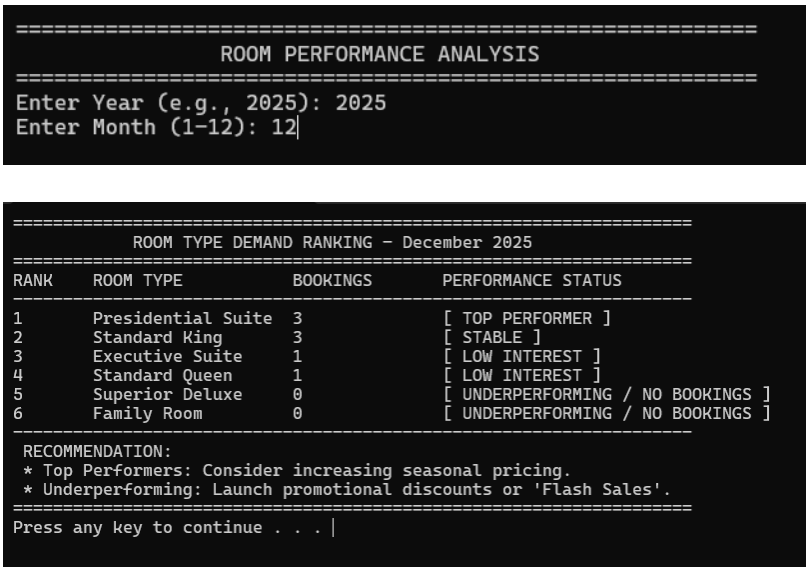
| | | |
|---|---|---|
| 3.4.23 |  <br>  | Delete user by adding keyword name or username will give option to delete then ask staff to confirm for delete user but it will show fail if the user have active booking. |
| 3.4.24 |  | Manage booking menu have option to add, search, delete, view, check in and check out for staff to enter. |

| | | |
|---|---|---|
| 3.4.25 |  | Add booking which asks staff to enter user id and fill details to book then asks to confirm the booking before display the successful booking. It will ask if the staff want to add another book. |
| 3.4.26 |  | For search booking, staff can verify customer booking id, customer name, room name, check in check out, price and status. It also asks staff if they want to search again. |

| | | |
|---|---|---|
| 3.4.27 |  | Cancel booking which ask staff to enter booking id and then choose option to cancel booking or delete booking then display customer that they want to cancel or delete. After that, it will ask to confirm cancel booking. |
| 3.4.28 |  | View all booking for staff see all details booking. |

| | | |
|---|---|---|
| 3.4.29 |  | Check in/check out function which ask staff to enter booking id then display selected id and status. Then it will show option to confirm check in or check out. |
| 3.4.30 |  | Report menu that shows all report option. |
| 3.4.31 |  | Room inventory and status report providing a summary of operational status, unit counts and maintenance alerts for effective facility management. |

| | | |
|---|---|---|
| 3.4.32 |  | Financial performance report providing revenue summary and revenue by category room. |
| 3.4.33 |  | Guest report providing total register and top tier member. |
| 3.4.34 |  | Monthly revenue trend analysis report providing revenue for each month. |

| | | |
|---|---|---|
| 3.4.35 |  | Monthly revenue visualization report providing a bar chart for each month. |
| 3.4.36 |  | Room performance analysis report providing ranking room and performance status. |

# CHAPTER 4: IMPLEMENTATION

## 4.1 Naming Convention

```
struct CurrentUser {
    int userId = 0;
    int specificId = 0;
    int roleId = 0;
    string username = "";
    string role = "";
    bool isLoggedIn = false;
} currentUser;


// --- Function Prototypes ---
int getLastInsertID(sql::Connection* con);
string getHiddenPassword();
bool safeIntInput(int& input);
string getRoleStringById(sql::Connection* con, int roleId);
long long daysBetweenDates(const string& date1, const string& date2);
string getFloorByID(int roomId);
int getValidChoice(int min, int max);
void refreshUserPoints(sql::Connection* con, int& points, string& level);
void registerCustomer(sql::Connection* con);
void registerUser(sql::Connection* con, bool isStaff);
bool loginUser(sql::Connection* con);
void customerDashboard(sql::Connection* con);
void staffDashboard(sql::Connection* con);
void adminRoomMenu(sql::Connection* con);
void adminUserMenu(sql::Connection* con);
void adminBookingMenu(sql::Connection* con);
void syncRoomStatus(sql::Connection* con, int roomId);
void manageRooms(sql::Connection* con);
void viewAvailableRooms(sql::Connection* con, bool showPause = true);
void searchRoom(sql::Connection* con);
void updateRoomDetails(sql::Connection* con);
void deleteRoom(sql::Connection* con);
void createUserAdmin(sql::Connection* con);
void viewAllUsers(sql::Connection* con, bool showPause);
void searchUser(sql::Connection* con);
void updateUserDetails(sql::Connection* con);
void deleteUser(sql::Connection* con);
void bookRoom(sql::Connection* con);
void checkMyRewards(sql::Connection* con);
void generateReport(sql::Connection* con);
void viewAllBookings(sql::Connection* con, bool showPause);
void viewMyBookings(sql::Connection* con, bool showPause);
void updateBooking(sql::Connection* con);
void deleteBooking(sql::Connection* con);
void searchBooking(sql::Connection* con);
void manageCheckInOut(sql::Connection* con);
void autoCleanupNoShow(sql::Connection* con);
void reportMenu(sql::Connection* con);
void roomReport(sql::Connection* con);
void bookingReport(sql::Connection* con);
void guestReport(sql::Connection* con);
void revenueAnalysisReport(sql::Connection* con);
void revenueBarChart(sql::Connection* con);
void saveReportToDB(sql::Connection* con, string type, int totalB, double totalR);
void roomPopularityReport(sql::Connection* con);
```

Figure 4.1 Naming Convention

Naming convention in this system is for function naming it start with small letter for first word and capital letter for second letter. For example, loginUser. For variable it starts with small letters such as username.

## 4.2 Function

```
// --- Function Prototypes ---
int getLastInsertID(sql::Connection* con);
string getHiddenPassword();
bool safeIntInput(int& input);
string getRoleStringById(sql::Connection* con, int roleId);
long long daysBetweenDates(const string& date1, const string& date2);
string getFloorByID(int roomId);
int getValidChoice(int min, int max);
void refreshUserPoints(sql::Connection* con, int& points, string& level);
void registerCustomer(sql::Connection* con);
void registerUser(sql::Connection* con, bool isStaff);
bool loginUser(sql::Connection* con);
void customerDashboard(sql::Connection* con);
void staffDashboard(sql::Connection* con);
void adminRoomMenu(sql::Connection* con);
void adminUserMenu(sql::Connection* con);
void adminBookingMenu(sql::Connection* con);
void syncRoomStatus(sql::Connection* con, int roomId);
void manageRooms(sql::Connection* con);
void viewAvailableRooms(sql::Connection* con, bool showPause = true);
void searchRoom(sql::Connection* con);
void updateRoomDetails(sql::Connection* con);
void deleteRoom(sql::Connection* con);
void createUserAdmin(sql::Connection* con);
void viewAllUsers(sql::Connection* con, bool showPause);
void searchUser(sql::Connection* con);
void updateUserDetails(sql::Connection* con);
void deleteUser(sql::Connection* con);
void bookRoom(sql::Connection* con);
void checkMyRewards(sql::Connection* con);
void generateReport(sql::Connection* con);
void viewAllBookings(sql::Connection* con, bool showPause);
void viewMyBookings(sql::Connection* con, bool showPause);
void updateBooking(sql::Connection* con);
void deleteBooking(sql::Connection* con);
void searchBooking(sql::Connection* con);
void manageCheckInOut(sql::Connection* con);
void autoCleanupNoShow(sql::Connection* con);
void reportMenu(sql::Connection* con);
void roomReport(sql::Connection* con);
void bookingReport(sql::Connection* con);
void guestReport(sql::Connection* con);
void revenueAnalysisReport(sql::Connection* con);
void revenueBarChart(sql::Connection* con);
void saveReportToDB(sql::Connection* con, string type, int totalB, double totalR);
void roomPopularityReport(sql::Connection* con);
```

Figure 4.2 Function

All these functions have been used inside this system as it will simplify and optimize the code of the system a lot. Each function represents each menu and inside each function will have create, view, search, update and delete to be called.

## 4.3 Array

```
cout << "  Period    | Revenue (RM) | Visualization (Scale: * = RM500)\n";
cout << "  ----------+--------------+-------------------------------------------\n";

string monthNames[] = { "", "Jan", "Feb", "Mar", "Apr", "May", "Jun",
                        "Jul", "Aug", "Sep", "Oct", "Nov", "Dec" };
```

Figure 4.3 Array

In this system, array is implemented in this system to store a predefined list of month names. The monthNames array stores string data type elements, enabling the system to map month integers (1-12) retrieved from the database to human-readable month names such as converting '1' to 'January'. Using an Array allows for direct data access via indexing without using for lengthy if-else or switch statements, which results in more efficient and organized code.

## 4.4 Selection

```
cout << "=================================================\n";
cout << "               CUSTOMER DASHBOARD\n";
cout << "=================================================\n";
cout << " Welcome, " << currentUser.username << "!\n";
cout << " Membership Tier : [" << levelFromDB << "]\n";
cout << " Reward Points   : " << pointsFromDB << " pts\n";
cout << "-------------------------------------------------\n";
cout << " 1. Book a Room\n";
cout << " 2. View Available Rooms\n";
cout << " 3. Check Rewards & Benefits\n";
cout << " 4. View My Booking History\n";
cout << " 5. Cancel a Booking\n";
cout << " 6. Logout\n";
cout << "-------------------------------------------------\n";
cout << "Enter your choice: ";

int choice;
if (safeIntInput(choice)) {
    switch (choice) {
    case 1:
        bookRoom(con);

        break;
    case 2:
        viewAvailableRooms(con, true);
        break;
    case 3:
        checkMyRewards(con);
        break;
    case 4:
        viewMyBookings(con, true);
        break;
    case 5:
        deleteBooking(con);
        break;
    case 6:
        currentUser.isLoggedIn = false;
        cout << "Logging out...\n";
        system("pause");
        break;
    default:
        cout << "Invalid choice. Please try again.\n";
        system("pause");
    }
}
```

Figure 4.4 Selection Switch Case and If-Else

In this system, I have implemented two types of selection structures: if-else and switch-case to manage how the program makes decisions. The switch-case structure is used for the main menu and registration types, where the user picks a number like 1, 2, or 3 to go to a specific function. For example, on the registration menu, case 1 starts booking a room while case 2 is for view available rooms. This makes the code very organized and easy to read for simple choices. On the other hand, the if-else structure is used for more complex checking, such as validating if the user entered a real number or a wrong character using the safeIntInput function. It is also used to check if a username already exists in the database before saving new data. Using both techniques together helps the system run smoothly because switch-case handles the navigation, while if-else handles the logic and prevents errors from wrong user inputs.

## 4.5 Control

```
int main() {
    sql::Driver* driver;
    sql::Connection* con;
    try {
        driver = get_driver_instance();
        con = driver->connect(server, username, password);
        con->setSchema(schema);
        cout << "Database connection successful!" << endl;
        system("pause");

        while (true) {
            system("cls");
            cout << "========================================\n";
            cout << "      THE HOTEL RESERVATION SYSTEM\n";
            cout << "========================================\n";
            cout << " 1. Register\n";
            cout << " 2. Login\n";
            cout << " 3. Exit\n";

            int choice = getValidChoice(1, 3);
            if (choice == -1) continue;

            switch (choice) {
            case 1: registerCustomer(con); break;
            case 2:
                if (loginUser(con)) {
                    if (currentUser.role == "Customer") customerDashboard(con);
                    else staffDashboard(con);
                }
                break;
            case 3: delete con; return 0;
            }
        }
    }
    catch (sql::SQLException& e) {
        cout << "Could not connect to server. Error: " << e.what() << endl;
        system("pause");
        exit(1);
    }
    return 0;
}
```

Figure 4.5 Control

The while loop repetition structure is used in this system to keep the program running so the user can perform many tasks in one session. In functions like main, the loop ensures that after a user finish booking a room or checking their rewards, the system does not close but instead returns to the main menu. For example, in the main function, the while(true) loop keeps asking the user to either register or login until they choose to exit. This control structure works together with switch-case and if-else logic to handle user inputs safely. The loop only stops when the user selects "Exit" or "Logout," which triggers a return or changes the login status to false. This makes the system very user-friendly because it allows a continuous flow of operations without having to restart the application every time

## 4.6 Pointer

```
double hoursLeft = difftime(time1, time2) / 3600.0;
double* hoursPtr = &hoursLeft;

double refund_pct = 0.0;

if (*hoursPtr >= 48.0) refund_pct = 1.0;
else if (*hoursPtr >= 12.0) refund_pct = 0.7;
else if (*hoursPtr >= 3.0) refund_pct = 0.4;
else if (*hoursPtr > 0.0) refund_pct = 0.1;
else refund_pct = 0.0;

double refund_amount = totalPrice * refund_pct;

cout << "\n  +------------------------------------------------------------+\n";
cout << "     CANCELLATION SUMMARY \n";
cout << "     Time remaining   : " << fixed << setprecision(1) << *hoursPtr << " hours" << endl;
cout << "     Guest Name       : " << customerName << endl;
cout << "     Original Price   : RM " << fixed << setprecision(2) << totalPrice << endl;
cout << "     REFUND RATE      : " << (refund_pct * 100) << "%" << endl;
cout << "     REFUND TOTAL     : RM " << refund_amount << endl;
cout << "  +------------------------------------------------------------+\n\n";
```

Figure 4.6 Pointer

In the deleteBooking function, I created a pointer called hoursPtr that points to the address of hoursLeft. By using the address-of operator (&), the pointer is linked to that specific spot in memory. Then, I used the dereference operator (*), for example in the line if (*hoursPtr >= 48.0), to grab the actual value from that address to calculate the refund percentage. This technique is important because it shows how to handle memory efficiently.

## 4.7 Error Handling

```
bool safeIntInput(int& input) {
    if (cin >> input) {
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        return true;
    }
    else {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        return false;
    }
}
```

Figure 4.7 Input validation

In this project, I have implemented Error Handling as a technique to manage user mistakes and ensure the system stays stable while it is running. The main example of this is the safeIntInput function, which is designed to handle "invalid inputs" from the user. For instance, if the system asks for a room ID (a number) but the user accidentally types a letter, the program would normally crash or act strangely. To prevent this, my code uses an if-else structure to check if the input is correct. If the input is wrong, the code uses cin.clear() to reset the input stream and cin.ignore() to delete the bad data. This allows the program to show an error message and ask the user to try again instead of stopping the whole system. This technique is very important for making a professional and user-friendly application because it makes the program "robust" against common human errors.

```
bool loginUser(sql::Connection* con) {
    int attempts = 0;
    const int MAX_ATTEMPTS = 3;

    while (attempts < MAX_ATTEMPTS) {
        system("cls");
        string user, pass;

        cout << "========================================\n";
        cout << "               USER LOGIN               \n";
        cout << "========================================\n";
        cout << "Username: "; cin >> user;
        cout << "Password: "; pass = getHiddenPassword();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        try {
            sql::PreparedStatement* pstmt;
            sql::ResultSet* res;

            pstmt = con->prepareStatement("SELECT User_ID, RoleID, Username, Password FROM USER WHERE Username = ?");
            pstmt->setString(1, user);
            res = pstmt->executeQuery();

            if (res->next()) {
                string storedPassword = res->getString("Password");

                if (pass == storedPassword) {
                    currentUser.userId = res->getInt("User_ID");
                    currentUser.roleId = res->getInt("RoleID");
                    currentUser.username = res->getString("Username");
                    currentUser.role = getRoleStringById(con, currentUser.roleId);

                    delete res; delete pstmt;

                    if (currentUser.role == "Customer") {
                        pstmt = con->prepareStatement("SELECT Customer_ID FROM CUSTOMER WHERE User_Id = ?");
                        pstmt->setInt(1, currentUser.userId);
                        res = pstmt->executeQuery();
                        if (res->next()) currentUser.specificId = res->getInt("Customer_ID");
                    }
                    else {
                        pstmt = con->prepareStatement("SELECT Staff_ID FROM STAFF WHERE User_ID = ?");
                        pstmt->setInt(1, currentUser.userId);
                        res = pstmt->executeQuery();
                        if (res->next()) currentUser.specificId = res->getInt("Staff_ID");
                    }

                    delete res; delete pstmt;
                    currentUser.isLoggedIn = true;

                    string dashboardType = (currentUser.role == "Customer") ? "Customer Dashboard" : "Staff Dashboard";

                    cout << "\n[SUCCESS] Login Successful! Welcome, " << currentUser.username << ".\n";
                    cout << "Logged in as: " << currentUser.role << " - Redirecting to " << dashboardType << ".\n\n";

                    system("pause");
                    return true;
                }
```

Figure 4.8 Error Handling

The system incorporates a robust error handling mechanism by implementing structured try-catch blocks throughout the source code. This approach is specifically designed to intercept and manage potential SQL exceptions, such as database connectivity failures and syntax execution errors, during runtime. By preventing abrupt system crashes, this mechanism ensures high system stability and provides more reliable user experience when interacting with the MySQL database.

## 4.8 Business Rule Implementation

```
string checkIn, checkOut;
cout << "Enter Check-In (YYYY-MM-DD): "; cin >> checkIn;
cout << "Enter Check-Out (YYYY-MM-DD): "; cin >> checkOut;

time_t t = time(0);
tm* now_ptr = localtime(&t);
tm tm_now = *now_ptr;

stringstream ss_today;
ss_today << (tm_now.tm_year + 1900) << "-"
    << setfill('0') << setw(2) << (tm_now.tm_mon + 1) << "-"
    << setfill('0') << setw(2) << tm_now.tm_mday;

long long days_till_checkin = daysBetweenDates(ss_today.str(), checkIn);
long long duration_days = daysBetweenDates(checkIn, checkOut);
```

```
if (days_till_checkin < 2) {
    cout << "\n[ERROR] Policy Violation: Reservations must be made at least 2 days in advance.\n";
    cout << "Today is: " << ss_today.str() << ". Please select a date at least 2 days away.\n";
    system("pause");
    continue;
}

if (duration_days <= 0) {
    cout << "\n[ERROR] Invalid duration. Check-Out must be after Check-In.\n";
    system("pause");
    continue;
}
```

Figure 4.9 Business Rule Implementation

In this system, I have implemented several Business Rules to automate hotel policies and ensure operational efficiency. The most critical business rule is the 2-Day Advance Booking Policy, which prevents guests from making last-minute reservations; the system calculates the difference between the current date and the check-in date, rejecting any request made less than 48 hours in advance.

## 4.9 Complex Calculation Implementation

```
time_t now_t = time(0);
tm tm_now = *localtime(&now_t);

tm tm_checkIn = { 0 };
istringstream ss_checkIn(checkInDateStr);
ss_checkIn >> get_time(&tm_checkIn, "%Y-%m-%d");


tm_checkIn.tm_hour = 15;
tm_checkIn.tm_min = 0;
tm_checkIn.tm_sec = 0;

tm_checkIn.tm_isdst = -1;
tm_now.tm_isdst = -1;

time_t time1 = mktime(&tm_checkIn);
time_t time2 = mktime(&tm_now);

double hoursLeft = difftime(time1, time2) / 3600.0;
double* hoursPtr = &hoursLeft;

double refund_pct = 0.0;

if (*hoursPtr >= 48.0) refund_pct = 1.0;
else if (*hoursPtr >= 12.0) refund_pct = 0.7;
else if (*hoursPtr >= 3.0) refund_pct = 0.4;
else if (*hoursPtr > 0.0) refund_pct = 0.1;
else refund_pct = 0.0;

double refund_amount = totalPrice * refund_pct;
```

Figure 4.10 Complex Calculation

The calculation for this system manages the hotel's automated refund and cancellation policy. This implementation is found in the cancellation module, where the program must calculate a guest's refund based on how much time is left before their check-in. The calculation begins by converting a simple date string into a structured time format and manually setting the check-in deadline to 3:00 PM to follow hotel rules. The system then uses specialized functions like mktime and difftime to find the exact difference in seconds between the current time and the deadline, which is then divided by 3600 to get the total hours remaining. To make the logic more advanced, a Pointer is used to access this value from memory to determine the refund percentage. The final complex step involves a graduated scale where the totalPrice is multiplied by different rates such as 100%, 70%, 40%, or 10% depending on the calculated hours. This ensures the hotel can automatically calculate fair refunds based on real-time data.

```
try {
    sql::Statement* stmt = con->createStatement();

    string query =
        "SELECT R.RoomType, R.RoomStatus, R.TotalQuantity, "
        "(SELECT COUNT(*) FROM BOOKING B WHERE B.Room_ID = R.Room_ID AND B.BookingStatus = 'Confirmed') AS BookedCount "
        "FROM ROOM R ORDER BY R.RoomType";

    sql::ResultSet* res = stmt->executeQuery(query);

    cout << " [ CURRENT INVENTORY STATUS ]\n\n";
```

Figure 4.11 Room Inventory and Status

In this system, complex SQL queries to pull data and make reports for roomReport. The program can run advanced SELECT commands to do more than just showing data. For example, in the Report Module, aggregate functions like SUM() and COUNT() with GROUP BY to turn raw booking records into monthly money reports. It also used Prepared Statements so the system can filter results easily, like checking which room is popular in a certain month or year. By joining tables like ROOM and BOOKING, the system gets enough data to calculate things like "Gained" or "Dropped" percentages and room popularity trends. This makes sure the information on the screen is accurate and helps the staff see how the business is doing.

```
cout << "-------------------------------------------------------------------------------------------------------------\n";
try {

    string query = (roleFilter == "Customer") ?
        "SELECT u.User_ID, u.Username, r.Position, u.User_Email, u.User_PhoneNumber, c.Name "
        "FROM USER u JOIN ROLE r ON u.RoleID = r.RoleID JOIN CUSTOMER c ON u.User_ID = c.User_Id" :
        "SELECT u.User_ID, u.Username, r.Position, u.User_Email, u.User_PhoneNumber, s.StaffName "
        "FROM USER u JOIN ROLE r ON u.RoleID = r.RoleID JOIN STAFF s ON u.User_ID = s.User_ID "
        "WHERE LOWER(r.Position) != 'customer'";

    sql::Statement* stmt = con->createStatement();
```

Figure 4.12 View All User

The code implements a dynamic query system to fetch user details from the database based on the selected roleFilter. By using a conditional statement, the system determines whether to execute a query for customers or staff members. When the filter is set to "Customer," the program joins the USER, ROLE, and CUSTOMER tables to retrieve specific information such as the customer's name, email, and phone number. Conversely, if the filter is not for a customer, the system joins the USER, ROLE, and STAFF tables instead. This second query includes a WHERE clause that converts the position to lowercase to ensure no customer roles are accidentally included in the staff list. Once the appropriate SQL string is constructed, a sql::Statement object is created to execute the command, allowing the system to display a filtered and organized list of users depending on their specific roles within the application.

## 4.10 Report Generation for Analysis Implementation

```
void revenueAnalysisReport(sql::Connection* con) {
    system("cls");
    cout << "============================================================\n";
    cout << "              MONTHLY REVENUE TREND ANALYSIS                \n";
    cout << "============================================================\n\n";

    try {
        sql::Statement* stmt = con->createStatement();
        sql::ResultSet* res = stmt->executeQuery(
            "SELECT YEAR(CheckIn_Date) AS Year, MONTH(CheckIn_Date) AS Month, SUM(TotalPrice) AS MonthlyRevenue "
            "FROM BOOKING WHERE BookingStatus != 'Cancelled' "
            "GROUP BY Year, Month ORDER BY Year, Month"
        );

        double previousRevenue = 0;
        string monthNames[] = { "", "JAN", "FEB", "MAR", "APR", "MAY", "JUN", "JUL", "AUG", "SEP", "OCT", "NOV", "DEC" };

        while (res->next()) {
            int year = res->getInt("Year");
            int month = res->getInt("Month");
            double currentRevenue = res->getDouble("MonthlyRevenue");

            cout << " [" << monthNames[month] << " " << year << "] ";
            cout << " Revenue: RM " << fixed << setprecision(2) << setw(10) << currentRevenue;

            if (previousRevenue > 0) {
                double pct = ((currentRevenue - previousRevenue) / previousRevenue) * 100;

                if (pct >= 0) {
                    cout << "   +" << setprecision(1) << pct << "% (Gained)";
                }
                else {
                    cout << "   " << setprecision(1) << pct << "% (Dropped)";
                }
            }
            else {
                cout << "  -- (Starting Point)";
            }
            cout << "\n------------------------------------------------------------\n";
            previousRevenue = currentRevenue;
        }

        cout << "\n >>> Quick Note: December peaks are standard due to holidays.\n";
        cout << " >>> Action: Focus on 'Jan-Feb' marketing to boost sales.\n";

        delete res; delete stmt;
    }
    catch (sql::SQLException& e) {
        cout << "Database Error: " << e.what() << endl;
    }
    system("pause");
}
```

Figure 4.10 Report Generation

The Report Generation for Analysis is implemented in this system to turn raw booking data into helpful information for the hotel manager. This feature is found in functions like revenueAnalysisReport, revenueBarChart, and roomPopularityReport. Instead of just showing a list of numbers, the system performs extra calculations to find patterns. For example, in the revenue report, the program saves the previous month's total in a variable and compares it to the current month to calculate a "Gained" or "Dropped" percentage. These automated reports are important because they give the hotel staff clear advice on how to improve their business based on real data.

# CHAPTER 5: CONCLUSION

## 5.1 Constraints

There are some constraints and limitations in developing this Hotel Reservation System. First, the system must focus on data accuracy. It is very important to make sure that room availability and guest details are always updated. This is to avoid problems like double booking, which can make the hotel management and guests unhappy. Other than that, the system has a limitation in terms of user access. Only authorized users like Manager and Staff can manage the hotel records and view reports. Guests can only use the system to make and check their own bookings for security reasons. Besides that, the system focuses on basic functions like CRUD (Create, Read, Update, and Delete). More advanced features like online payment and high-level data analytics are not included in the current version of this project.

## 5.2 Future Improvements

Future improvements for The Hotel Reservation System could include an Automated Email or SMS Notification. Currently, users need to log in to check their booking status. By adding this feature, the system can automatically send a confirmation message to the guest's email or phone after they finish a booking or when their check-in date is near. This will provide better experience for the customers. Moreover, the system can improve the search function by adding filters like price range or room facilities. For example, a guest can search for rooms that are "Below RM200" or rooms that have "WiFi and Breakfast". This will make the system more helpful and save time for the users when looking for a specific type of room.

# REFERENCES

Bemile, R., Achampong, A., & Danquah, E. (2014). Online hotel reservation system. *International Journal of Innovative Science, Engineering & Technology*, *1*(9), 583-588.

Koh, W. S., & Hassim, Y. M. M. (2021). Hotel reservation management system. *Applied Information Technology And Computer Science*, *2*(2), 973-992.

Abdulrauf, A., Ogar, A., Delmut, R. D., & Chidiebere, O. (2023). Development of a Web-Based Reservation Systems for Federal Polytechnic Kaltungo. *Development*, *7*(7), 25-38.

MySQL Documentation. (n.d.). Retrieved from https://dev.mysql.com/doc/